Natural Language Processing

Word Representations

Fall 2025/26





Based on slides from Mor Geva, Maor Ivgi, Jonathan Berant, Richard Socher, Chris Manning, and Hugo Lachorelle

Announcements

- **Hi, I'm Tal Wagner**
  - I teach this class on other semesters
  - I'm filling in for Mor this week

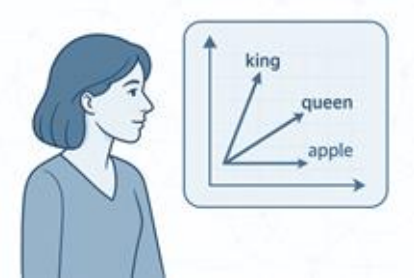
- This is a one-time classroom
- Starting next week you'll be in Holtzblat Hall (אולם הולצבלט)

Today's Plan

- **Word representations:**
  - Recap: Skipgram
  - Skipgram with negative sampling
  - Relation to matrix factorization
  - Evaluating word representations
- **Language modeling:**
  - We will start if we have time

Word Vectors: Recap

- **Goal**: Represent words as vectors s.t. inner product captures semantic similarity $\langle v, u \rangle = \sum_i v_i u_i = \|u\| \|v\| \cos \theta_{u,v}$
- We saw three methods:
  - One-hot vectors
  - Rows of co-occurrence matrix
  - Word2vec – skipgram



$$\text{Father} = \begin{bmatrix} 0.278 \\ 0.545 \\ 0.355 \\ 0.002 \\ -0.048 \end{bmatrix} \quad \text{Mother} = \begin{bmatrix} 0.298 \\ -0.615 \\ 0.355 \\ 0.003 \\ -0.051 \end{bmatrix} \quad \text{Dad} = \begin{bmatrix} 0.268 \\ 0.355 \\ 0.555 \\ 0.003 \\ -0.051 \end{bmatrix}$$

Word representation – discrete vectors

- Define a vocabulary $\mathcal{V}$ – ordered set of words, $|\mathcal{V}| = C$
- A word will be represented by $e_i \in \{0,1\}^C$ One-hot encoding

$\mathcal{V}$={Hello, World, Tel-Aviv}

Hello := [1,0,0]
World := [0,1,0]
Tel-Aviv := [0,0,1]

- What is the size of such vocabulary?
- How would we represent a sentence?

  Bag of words: “Hello World” → [1,1,0]

  - What would happen to the order? repetitions?

One-Hot Word Vectors: Pros & Cons

Pros:

One-Hot Word Vectors: Pros & Cons

Pros:

- Natural way to compose words into sentences
  - (“Natural” does not mean “ideal”: e.g., it loses word ordering)
- **Dot-product** between two sentence vectors = **size of lexical overlap**

One-Hot Word Vectors: Pros & Cons

Pros:

- Natural way to compose words into sentences
  - (“Natural” does not mean “ideal”: e.g., it loses word ordering)
- **Dot-product** between two sentence vectors = **size of lexical overlap**

Cons:

One-Hot Word Vectors: Pros & Cons

Pros:

- Natural way to compose words into sentences
  - (“Natural” does not mean “ideal”: e.g., it loses word ordering)
- **Dot-product** between two sentence vectors = **size of lexical overlap**

Cons:

- No semantic similarity between words
  - All word vectors are pairwise orthogonal
  - a great movie ≈ a great film ; a great movie ≉ a great meal
  - Ideally we want:

$$movie \cdot movie > movie \cdot film \gg movie \cdot meal$$

Distributional Semantics

Hypothesis: Context distribution captures word meaning

- Key insight: "You shall know a word by the company it keeps" (J. R. Firth 1957)
- Given a **word** *w* in a text, define its **context** as the set of words that appear nearby (within a fixed-size widow).

"… cashed a check at the **bank** across the street "
"… played happily on the **bank**, watching the ducks swim by "
"… adjusted the controls on the **bank** to dim the lights in the auditorium "
"… she will **bank** on her quick thinking to overcome the challenges "
"… the hospital requested donations to replenish their blood **bank** "
"… she made a **bank** shot, pocketing the eight ball in the corner "
"… the pilot needed to **bank** the plane sharply to the right "

Co-occurrence Matrix

Coordinate per each vocabulary word

- Given a large corpus, form the matrix of word co-occurrence counts

	cinema	tickets	star	chef	review	entree
movie	14	12	27	3	10	0
film	13	15	29	2	9	1
restaurant	3	1	14	33	12	29

- The vector representation of a word is its row in the matrix
  - Better: take the log-counts **(why?)**
  - Better: normalize the rows **(why?)**

$$\langle v, u \rangle = \sum_i v_i u_i = \|u\| \|v\| \cos \theta_{u,v}$$

Co-occurrence Matrix: Pros & Cons

Pros:

Co-occurrence Matrix: Pros & Cons

Pros:

- Capture semantics (to an extent)
  - E.g., synonyms would have similar context distributions → similar vectors

Co-occurrence Matrix: Pros & Cons

Pros:

- Capture semantics (to an extent)
  - E.g., synonyms would have similar context distributions → similar vectors

Cons:

Co-occurrence Matrix: Pros & Cons

Pros:

- Capture semantics (to an extent)
  - E.g., synonyms would have similar context distributions → similar vectors

Cons:

- **Efficiency**: dimension is huge (as large as vocabulary size)
- **Generalization?** Basically memorizes corpus

Skipgram (word2vec): Recap

Supervised learning - reminder

- Input: training set

$$\{(x_i, y_i)\}_{i=1}^{N},\ (x_i, y_i) \sim D(\mathcal{X} \times \mathcal{Y})$$

- Output: probabilistic model

$$f : \mathcal{X} \to \mathcal{Y}$$

$$\arg\max_y p(y \mid x)$$

Self-Supervised Learning

- Collect unlabeled data
- Create synthetic “proxy” task (potentially useless on its own)
- Hope it spurs the model to learn something useful from the data
- Advantage: no manual labeling needed

Self-Supervised Learning

- Collect unlabeled data
- Create synthetic “proxy” task (potentially useless on its own)
- Hope it spurs the model to learn something useful from the data
- Advantage: no manual labeling needed
- Example: masking

The fox jumped over the fence



The fox _____ over the fence

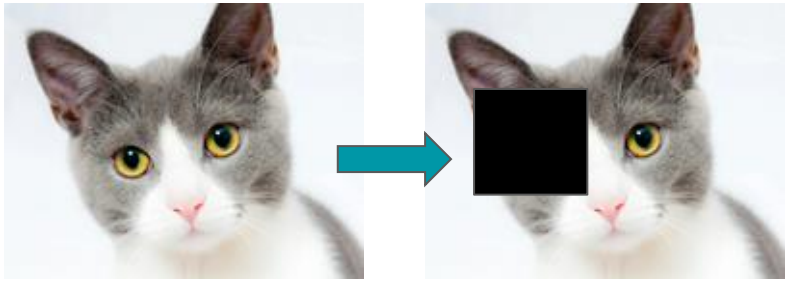
Self-Supervised Learning

- Collect unlabeled data
- Create synthetic “proxy” task (potentially useless on its own)
- Hope it spurs the model to learn something useful from the data
- Advantage: no manual labeling needed
- Example: masking

The fox jumped over the fence



The fox _____ over the fence



Word2Vec (Skipgram)

TL;DR: predict words from neighboring words

Mikolov et al., 2013

Word2Vec (Skipgram)

TL;DR: predict words from neighboring words

- Define self-supervised learning task from raw data:
  - Input: “center” word, label: “outside” (context) word

“… that bank holds the mortgage on my home…”



1. (x, y) = (bank, that)
2. (x, y) = (bank, holds)
3. (x, y) = (holds, bank)
4. (x, y) = (holds, the)
5. (x, y) = (the, holds)
6. (x, y) = (the, mortgage)
7. (x, y) = (mortgage, the)
8. (x, y) = (mortgage, on)
9. (x, y) = (on, mortgage),
10. (x, y) = (on, my)
11. (x, y) = (my, on)
12. (x, y) = (my, home)

Mikolov et al., 2013

Word2Vec (Skipgram)

TL;DR: predict words from neighboring words

- Define self-supervised learning task from raw data:
  - Input: "center" word, label: "outside" (context) word
- Define model for predicting label from input:

Example:
p("holds" | "bank")

$$p_\theta(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T v_c)}$$

"softmax"

Dot product compares similarity of o and c. Exponent makes everything positive

Normalize over the entire vocabulary to give a probability distribution

Parameters θ: (how many?) $|\theta| = 2 \cdot |\mathcal{V}| \cdot d$

$u_o \in \mathbb{R}^d$ vector for "outside" word(s) (context) o

$v_c \in \mathbb{R}^d$ vector for "center" word c

Mikolov et al., 2013

Word2Vec (Skipgram)

TL;DR: predict words from neighboring words

- Define self-supervised learning task from raw data:
  - Input: “center” word, label: “outside” (context) word
- Define model for predicting label from input:
- Define objective function for corpus of length T and window m

Objective to maximize: $$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{-m \leq j \leq m,\ j \neq 0} p_\theta(w_{t+j} | w_t)$$

Loss to minimize: $$\mathcal{J}(\theta) = -\log(\mathcal{L}(\theta))$$

Negative log-likelihood $$= \sum_{t=1}^{T} \sum_{-m \leq j \leq m,\ j \neq 0} \log\left(p_\theta(w_{t+j} | w_t)\right)$$

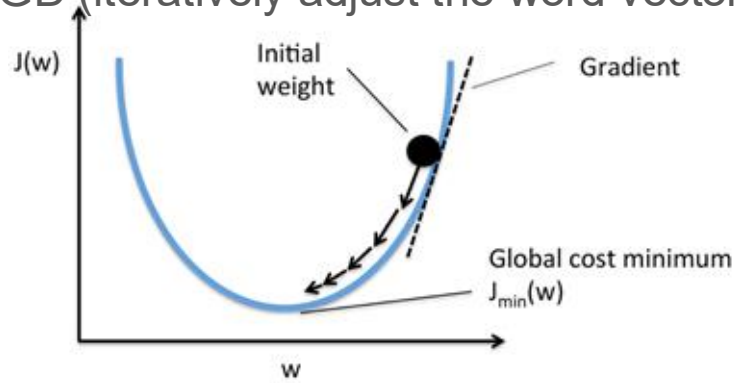
Mikolov et al., 2013

Word2Vec (Skipgram)

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m,\ j \ne 0} p_\theta(w_{t+j} | w_t)$$

TL;DR: predict words from neighboring words

- Define self-supervised learning task from raw data:
  - Input: “center” word, label: “outside” (context) word
- Define model for predicting label from input:
- Define objective function for corpus of length T and window m
- Optimize with SGD (iteratively adjust the word vectors to minimize NLL)

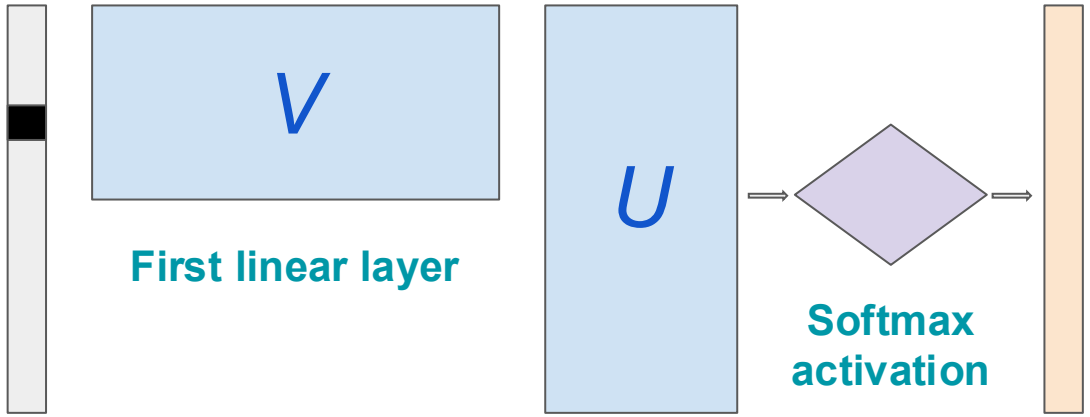


Mikolov et al., 2013

$$p_\theta(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T v_c)}$$

Skipgram: Neural Network

An equivalent way to look at the word2vec model:



Input:
One-hot encoding of center word

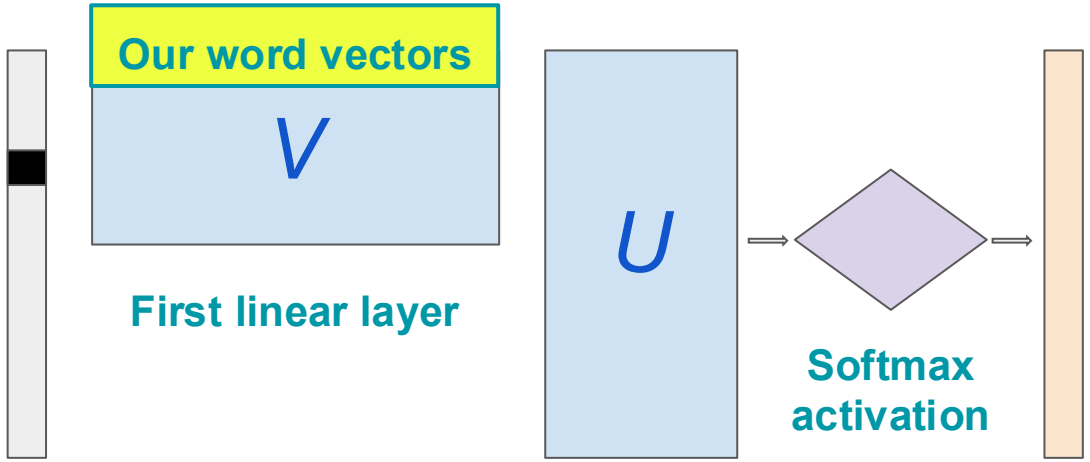
Second linear layer

Output:
Vector of outside word probabilities

$$p_\theta(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T v_c)}$$

Skipgram: Neural Network

An equivalent way to look at the word2vec model:



The columns of ***V*** are the center word vectors: v_c

The rows of ***U*** are the outside word vectors: u_o^T

We don't care about the network's output, but about its **internal representations**

Skipgram: Visual Demo

https://talwagner.github.io/teaching/word2vec.html

Word2Vec (Skipgram)

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m,\ j \ne 0} p_\theta(w_{t+j} | w_t)$$

TL;DR: predict words from neighboring words

- Define self-supervised learning task from raw data:
  - Input: “center” word, label: “outside” (context) word
- Define model for predicting label from input:
- Define objective function for corpus of length T and window m

What probabilities would maximize the objective (minimize the loss)?

Mikolov et al., 2013

Word2Vec (Skipgram)

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m,\ j \ne 0} p_\theta(w_{t+j} | w_t)$$

TL;DR: predict words from neighboring words

- Define self-supervised learning task from raw data:
  - Input: "center" word, label: "outside" (context) word
- Define model for predicting label from input:
- Define objective function for corpus of length T and window m

What probabilities would maximize the objective (minimize the loss)?

$$p(o|c) = \frac{\#(c,o)}{\sum_{o'} \#(c,o')}$$

The empirical distribution

#(c,o) is the number of co-occurrences of o and c.

Mikolov et al., 2013

Word2Vec (Skipgram)

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m,\ j \ne 0} p_\theta(w_{t+j} | w_t)$$

TL;DR: predict words from neighboring words

- Define self-supervised learning task from raw data:
  - Input: "center" word, label: "outside" (context) word
- Define model for predicting label from input:
- Define objective function for corpus of length T and window m

What probabilities would maximize the objective (minimize the loss)?

$$p(o|c) = \frac{\#(c,o)}{\sum_{o'} \#(c,o')}$$

The empirical distribution

#(c,o) is the number of co-occurrences of o and c.

Why don't we just choose vectors s.t. $e^{u_o^T v_c}$ = #(c,o) instead of learning?

Mikolov et al., 2013

Word2Vec (Skipgram)

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m,\ j \ne 0} p_\theta(w_{t+j} | w_t)$$

TL;DR: predict words from neighboring words

- Define self-supervised learning task from raw data:
  - Input: "center" word, label: "outside" (context) word
- Define model for predicting label from input:
- Define objective function for corpus of length T and window m

What probabilities would maximize the objective (minimize the loss)?

$$p(o|c) = \frac{\#(c,o)}{\sum_{o'} \#(c,o')}$$

The empirical distribution

#(c,o) is the number of co-occurrences of o and c.

Why don't we just choose vectors s.t. $e^{u_o^T v_c}$ = #(c,o) instead of learning?
Do such vectors exist? (Hint: the answer is "it depends" - on what?)

Mikolov et al., 2013

Skipgram vs. Co-occurrence Matrix

- The NLL loss is minimized when the model probabilities are equal to the empirical probabilities in our corpus: $p(o|c) = \frac{\#(c,o)}{\sum_{o'} \#(c,o')}$
- If the dimension ***d*** is equal to the vocabulary size, we can achieve this by simply setting:
  - **V** = log-co-occurrence matrix
  - **U** = unit matrix
- Hence:
  - Skipgram with ***d* = |*Vocab*|** reduces to co-occurrence matrix
  - Skipgram with ***d* << |*Vocab*|** acts as **regularization**

Skipgram: Additional Points

- What dimension ***d*** do we actually choose?

Skipgram: Additional Points

- What dimension ***d*** do we actually choose?
- Why have two vectors per word, **V** and **U**, not just one?

Skipgram: Additional Points

- What dimension ***d*** do we actually choose?
- Why have two vectors per word, **V** and **U**, not just one?

Pr[“gold” | “medal”] vs. Pr[“medal” | “gold”]

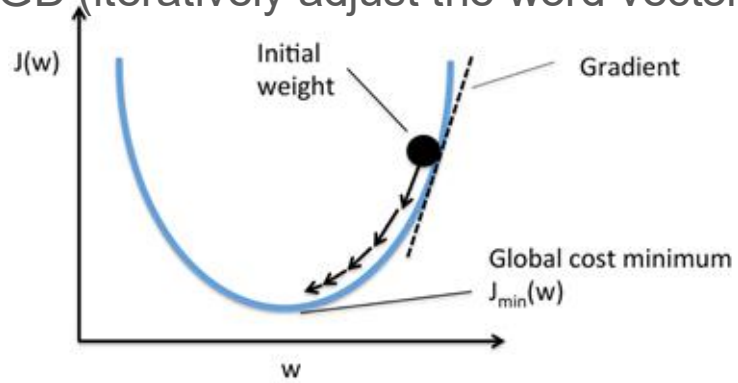
Skipgram with Negative Sampling

Word2Vec (Skipgram)

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m,\ j \ne 0} p_\theta(w_{t+j} | w_t)$$

TL;DR: predict words from neighboring words

- Define self-supervised learning task from raw data:
  - Input: “center” word, label: “outside” (context) word
- Define model for predicting label from input:
- Define objective function for corpus of length T and window m
- Optimize with SGD (iteratively adjust the word vectors to minimize NLL)



Mikolov et al., 2013

Skipgram: Loss Gradient

$$\log p(o|c) = \log \frac{\exp(u_o^T v_c)}{\sum_{o'} \exp(u_{o'}^T v_c)}$$

$$\nabla_{v_c} \log p(o|c) = u_o - \frac{1}{\sum_{o'} \exp\left(u_{o'}^T v_c\right)} \cdot \sum_{o'} \exp\left(u_{o'}^T v_c\right) \cdot u_{o'}$$

$$= u_o - \sum_{o'} \frac{\exp\left(u_{o'}^T v_c\right)}{\sum_{o''} \exp\left(u_{o''}^T v_c\right)} \cdot u_{o'}$$

$$= u_o - \sum_{o'} p(o'|c) \cdot u_{o'} = u_o - E_{o' \sim p(o'|c)}[u_{o'}]$$

Skipgram: Loss Gradient

$$\log p(o|c) = \log \frac{\exp(u_o^T v_c)}{\sum_{o'} \exp(u_{o'}^T v_c)}$$

$$\nabla_{v_c} \log p(o|c) = u_o - \frac{1}{\sum_{o'} \exp(u_{o'}^T v_c)} \cdot \sum_{o'} \exp(u_{o'}^T v_c) \cdot u_{o'}$$

$$= u_o - \sum_{o'} \frac{\exp(u_{o'}^T v_c)}{\sum_{o''} \exp(u_{o''}^T v_c)} \cdot u_{o'}$$

$$= u_o - \sum_{o'} p(o'|c) \cdot u_{o'} = u_o - E_{o' \sim p(o'|c)}[u_{o'}]$$

See an issue here?

Skipgram with negative sampling

- Same overall idea as the Skipgram we've seen so far, but:
  - We change the self-supervised learning task
  - And the probabilistic model
  - And the objective function

Mikolov et al., 2013

Negative Sampling

Binary classification instead of multi-class classification

“… that bank holds the mortgage on my home…”

Mikolov et al., 2013

Negative Sampling

Binary classification instead of multi-class classification

"… that bank holds the mortgage on my home…"

1. (x, y) = (bank, that)
2. (x, y) = (bank, holds)
3. (x, y) = (holds, bank)
4. (x, y) = (holds, the)
5. (x, y) = (the, holds)
6. (x, y) = (the, mortgage)
7. (x, y) = (mortgage, the)
8. …

Mikolov et al., 2013

Negative Sampling

Binary classification instead of multi-class classification

"… that bank holds the mortgage on my home…"

1. (x, y) = (bank, that)
2. (x, y) = (bank, holds)
3. (x, y) = (holds, bank)
4. (x, y) = (holds, the)
5. (x, y) = (the, holds)
6. (x, y) = (the, mortgage)
7. (x, y) = (mortgage, the)
8. …



1. (x, y) = ((bank, that), 1)
2. (x, y) = ((bank, table), 0)
3. (x, y) = ((bank, eat), 0)
4. (x, y) = ((holds, bank), 1)
5. (x, y) = ((holds, quickly), 0)
6. (x, y) = ((holds, which), 0)
7. …

Mikolov et al., 2013

Skipgram with negative sampling

Intuition: Increase the probability of pairs we saw and decrease the probability for everything else. Why?

Mikolov et al., 2013

Skipgram with negative sampling

Intuition: Increase the probability of pairs we saw and decrease the probability for everything else. Why?

$$p_\theta(y = 1|c, o) = \frac{1}{1+\exp(-u_o^T v_c)} = \sigma(u_o^T v_c)$$

$$p_\theta(y = 0|c, o) = 1 - \sigma(u_o^T v_c) = \sigma(-u_o^T v_c)$$

Mikolov et al., 2013

Skipgram with negative sampling

Intuition: Increase the probability of pairs we saw and decrease the probability for everything else. Why?

$$p_\theta(y=1|c,o) = \frac{1}{1+\exp(-u_o^T v_c)} = \sigma(u_o^T v_c)$$

$$p_\theta(y=0|c,o) = 1 - \sigma(u_o^T v_c) = \sigma(-u_o^T v_c)$$

Skipgram's model (reminder):

$$p_\theta(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T v_c)}$$

Mikolov et al., 2013

Skipgram with negative sampling

Intuition: Increase the probability of pairs we saw and decrease the probability for everything else. Why?

$$p_\theta(y=1|c,o) = \frac{1}{1+\exp(-u_o^T v_c)} = \sigma(u_o^T v_c)$$

$$p_\theta(y=0|c,o) = 1 - \sigma(u_o^T v_c) = \sigma(-u_o^T v_c)$$

Skipgram's model (reminder):

$$p_\theta(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T v_c)}$$

$$\mathcal{L}'(\theta) = \sum_{t=1}^{T} \left[\sum_{\substack{-m \le j \le m \\ j \neq 0}} \left(\log\left(\sigma\left(u_{w_{t+j}}^T v_{w_t} \right) \right) \right) \right]$$

Pairs we saw (positive samples)

Mikolov et al., 2013

Skipgram with negative sampling

Intuition: Increase the probability of pairs we saw and decrease the probability for everything else. Why?

$$p_\theta(y=1|c,o) = \frac{1}{1+\exp(-u_o^T v_c)} = \sigma(u_o^T v_c)$$

$$p_\theta(y=0|c,o) = 1 - \sigma(u_o^T v_c) = \sigma(-u_o^T v_c)$$

Skipgram's model (reminder):

$$p_\theta(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w\in\mathcal{V}} \exp(u_w^T v_c)}$$

$$\mathcal{L}'(\theta) = \sum_{t=1}^{T} \left[\sum_{\substack{-m\le j\le m \\ j\neq 0}} \left(\log\left(\sigma\left(u_{w_{t+j}}^T v_{w_t}\right)\right)\right) + \sum_{k\sim p(w)} \left(\log\left(\sigma\left(-u_{w^{(k)}}^T v_{w_t}\right)\right)\right) \right]$$

$p(w) = \frac{U(w)^{3/4}}{T}$

Pairs we saw (positive samples)

k random words (negative samples)

Mikolov et al., 2013

Contrastive learning

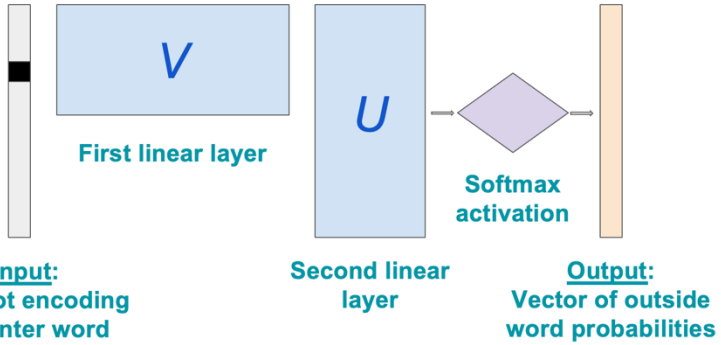
- Skipgram with negative sampling is an instance of contrastive learning
  - Data is pairs (x,y) that are similar to each other
  - You can assume that random pairs should be dissimilar

- Very popular nowadays for representation learning in vision (CLIP) and for information retrieval (DPR)

Word2Vec as a Neural Network

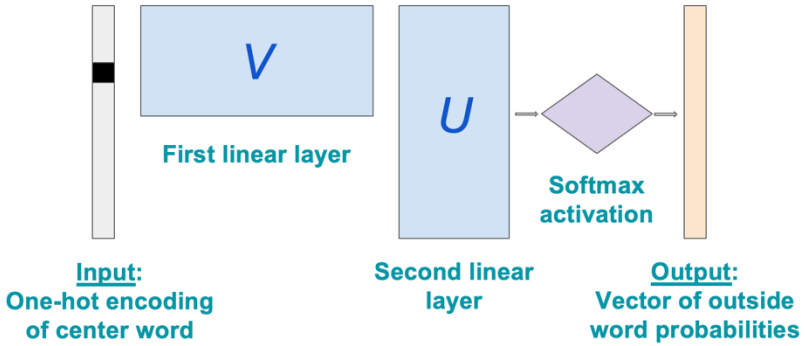
- **Skipgram:** $p_\theta(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T v_c)}$

- input = center word c
- output = $softmax(U^T V e_c)$



Word2Vec as a Neural Network

- **Skipgram:** $p_\theta(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T v_c)}$
- input = center word c
- output = $softmax(U^T V e_c)$



- **Skipgram with negative sampling:**

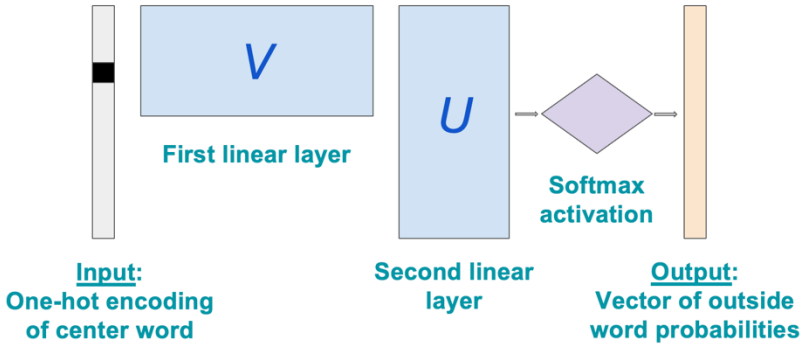
$$p_\theta(y = 1|c, o) = \frac{1}{1+\exp(-u_o^T v_c)} = \sigma(u_o^T v_c)$$

- input = center word c and outside word o
- output = ?

pe.app/talwagner

Word2Vec as a Neural Network

- **Skipgram:** $p_\theta(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T v_c)}$
- input = center word c
- output = $softmax(U^T V e_c)$



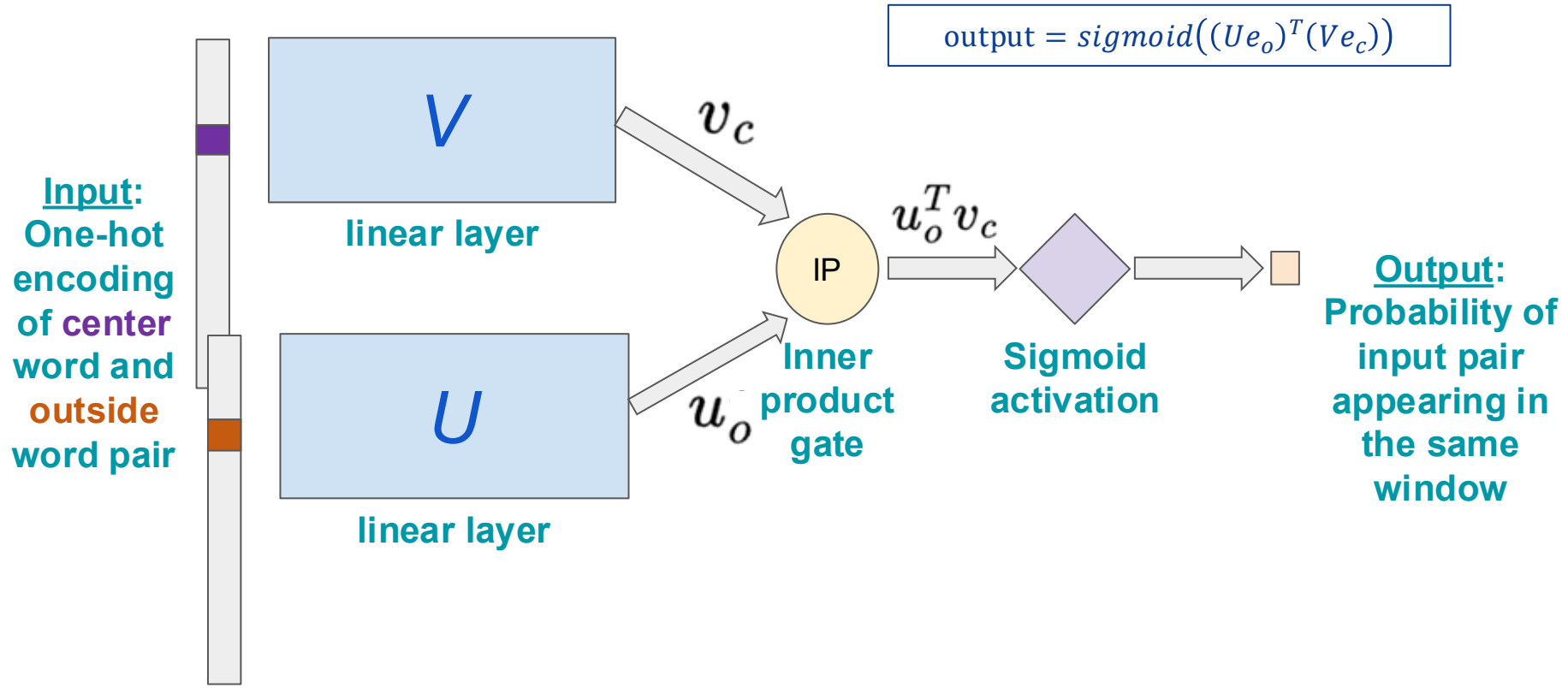
- **Skipgram with negative sampling:**

$$p_\theta(y = 1|c, o) = \frac{1}{1+\exp(-u_o^T v_c)} = \sigma(u_o^T v_c)$$

- input = center word c and outside word o
- output = ?

?

Skipgram with Negative Sampling: Neural Network



Word Representation: Key Takeaways

- Learning from raw data >> hard-coded human “knowledge”

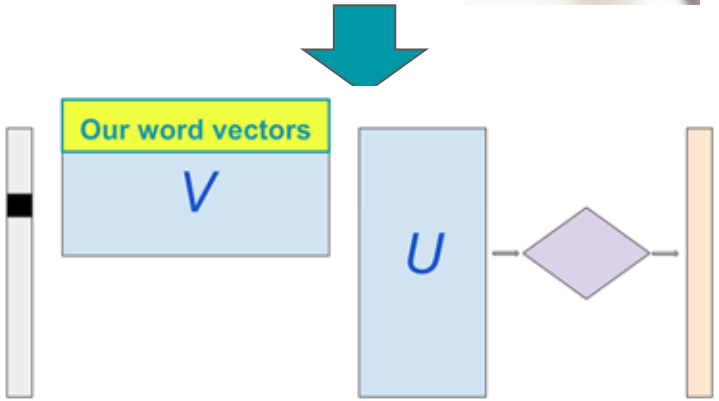
Word Representation: Key Takeaways

- Learning from raw data >> hard-coded human “knowledge”
- Learning representations with self-supervised learning:

The fox jumped over the fence

The fox _____ over the fence



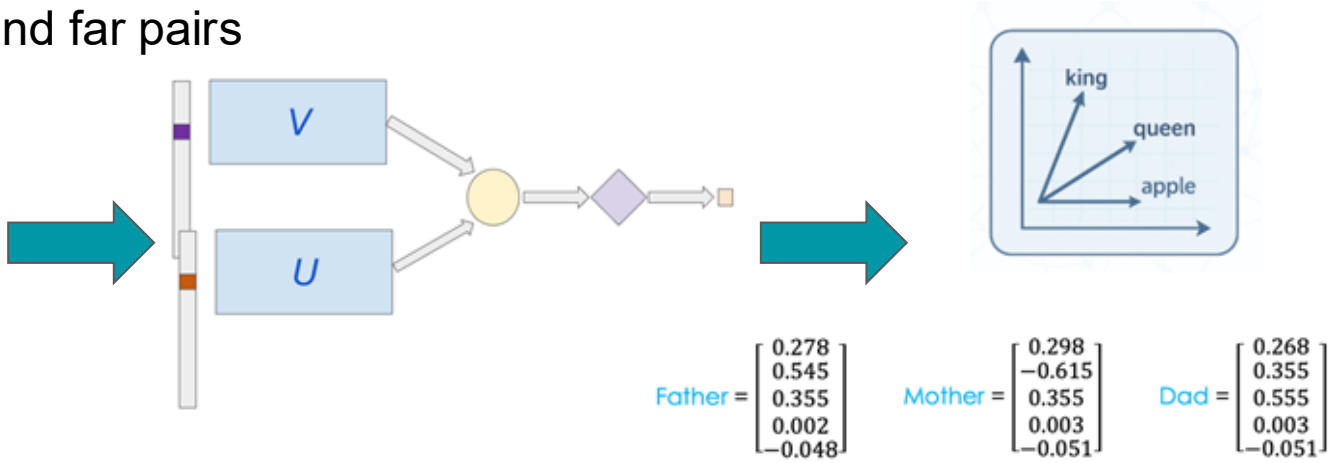


Word Representation: Key Takeaways

- Learning from raw data >> hard-coded human “knowledge”
- Learning representations with self-supervised learning
- Contrastive learning: Learn vector representation from binary examples of close pairs and far pairs

Close examples:
king-queen, car-road, movie-cinema, ...

Far examples:
king-ladybug, car-cabbage, flamenco-snow, …



Relation to matrix factorization

Recall: Skipgram vs. Co-occurrence Matrix

- If $d = |Vocab|$ then setting $\forall c, o\colon\ v_c^T u_o = \log \#(c, o)$ is optimal for Skipgram

Recall: Skipgram vs. Co-occurrence Matrix

- If $d = |Vocab|$ then setting $\forall c, o\colon\ v_c^T u_o = \log \#(c, o)$ is optimal for Skipgram
- If $d \ll |Vocab|$, intuitively, the model still tries to achieve this:

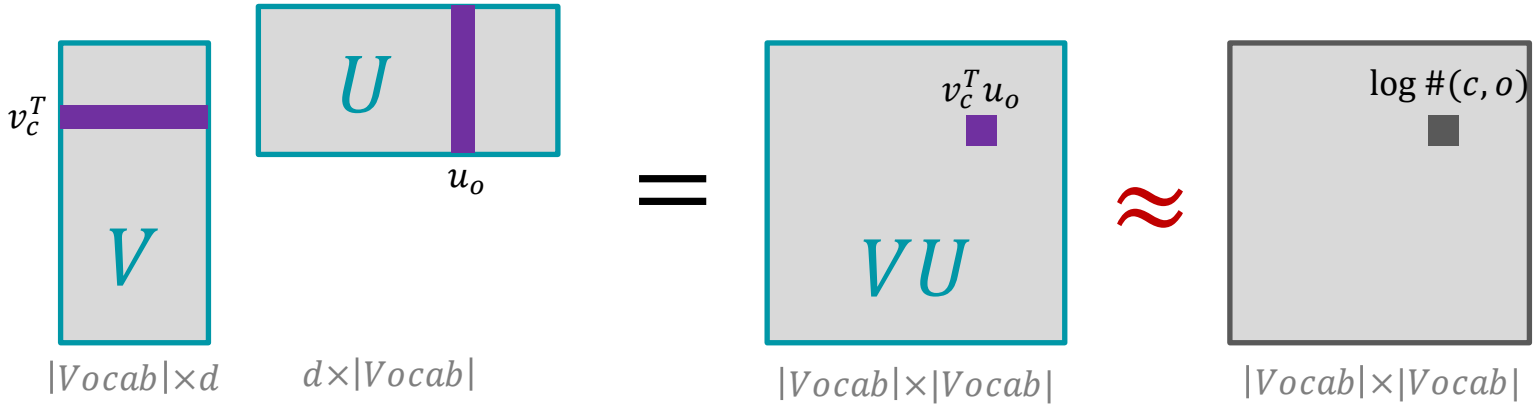
$$\forall c, o\colon\ v_c^T u_o \approx \log \#(c, o)$$

Recall: Skipgram vs. Co-occurrence Matrix

- If $d = |Vocab|$ then setting $\forall c, o\colon\ v_c^T u_o = \log \#(c, o)$ is optimal for Skipgram
- If $d \ll |Vocab|$, intuitively, the model still tries to achieve this:

$$\forall c, o\colon\ v_c^T u_o \approx \log \#(c, o)$$

- We can write this in matrix form:



Matrix Factorization

- Skipgram can thus be seen as **implicit matrix factorization**:
- Given a log-co-occurrence matrix M, find lower-rank U, V matrices such that



Matrix Factorization

- Skipgram can thus be seen as **implicit matrix factorization**:
- Given a log-co-occurrence matrix M, find lower-rank U, V matrices such that



- Question: what matrix does Skipgram with negative sampling implicitly factorize?

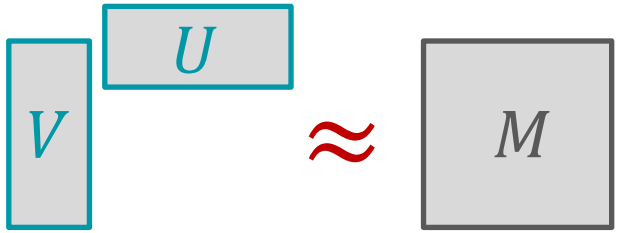
Matrix Factorization

- Skipgram can thus be seen as **implicit matrix factorization**:
- Given a log-co-occurrence matrix M, find lower-rank U, V matrices such that



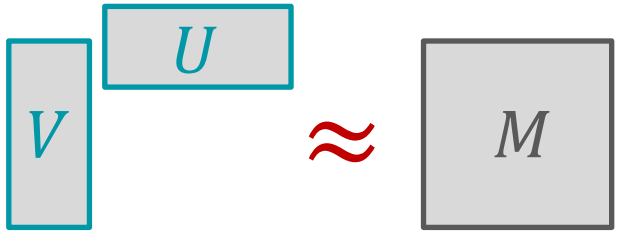
- Question: what matrix does Skipgram with negative sampling implicitly factorize?
  - Answer: the "shifted pointwise mutual information matrix" $M_{ij} = \log\left(\frac{k \cdot \#(c,o)}{|D| \cdot \#c \cdot \#o}\right)$
  - Levy & Goldberg, "Neural Word Embedding as Implicit Matrix Factorization", NeurIPS 2014

Matrix Factorization



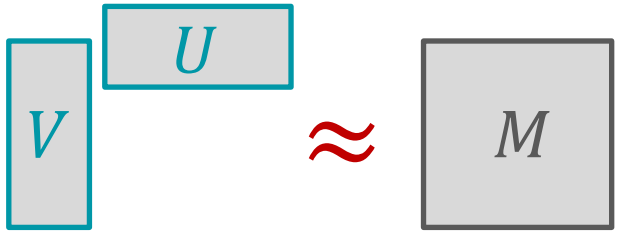
- Do we know methods for **explicit** matrix factorization?
  - Input: matrix M of order $n \times n$, and lower dimension $d \ll n$
  - Desired output: matrices V, U of order $n \times d, d \times n$ such that $VU \approx M$

Matrix Factorization



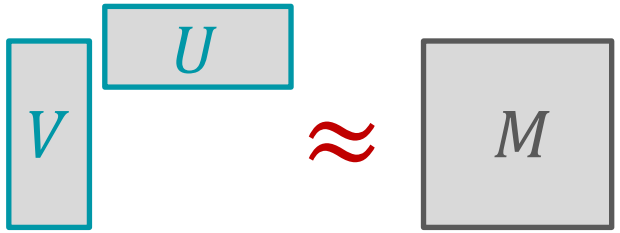
- Do we know methods for **explicit** matrix factorization?
  - Input: matrix M of order $n \times n$, and lower dimension $d \ll n$
  - Desired output: matrices V, U of order $n \times d, d \times n$ such that $VU \approx M$
- **Yes**: PCA, SVD…

Matrix Factorization



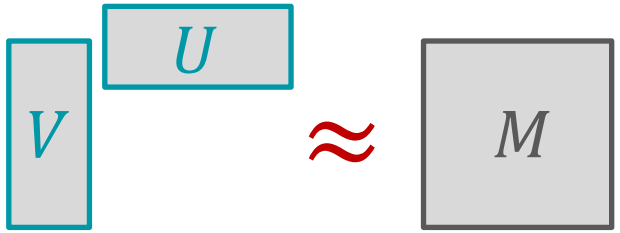
- Do we know methods for **explicit** matrix factorization?
  - Input: matrix M of order $n \times n$, and lower dimension $d \ll n$
  - Desired output: matrices V, U of order $n \times d, d \times n$ such that $VU \approx M$
- **Yes**: PCA, SVD…
  - Can we use them for word embeddings instead of word2vec?

Matrix Factorization



- Do we know methods for **explicit** matrix factorization?
  - Input: matrix M of order $n \times n$, and lower dimension $d \ll n$
  - Desired output: matrices V, U of order $n \times d, d \times n$ such that $VU \approx M$
- **Yes**: PCA, SVD…
  - Can we use them for word embeddings instead of word2vec?
  - Apparently, sometimes: *"factorization with SVD can achieve solutions that are at least as good as SGNS [for some tasks]"* (Levy & Goldberg, 2014)

Matrix Factorization



- Do we know methods for **explicit** matrix factorization?
  - Input: matrix M of order $n \times n$, and lower dimension $d \ll n$
  - Desired output: matrices V, U of order $n \times d, d \times n$ such that $VU \approx M$
- **Yes**: PCA, SVD…
  - Can we use them for word embeddings instead of word2vec?
  - Apparently, sometimes: *"factorization with SVD can achieve solutions that are at least as good as SGNS [for some tasks]"* (Levy & Goldberg, 2014)
- Can we combine the word2vec self-supervised learning idea with the matrix factorization view to get even better word embeddings?

GloVe Word Embeddings [Pennington, Socher, Manning 2014]

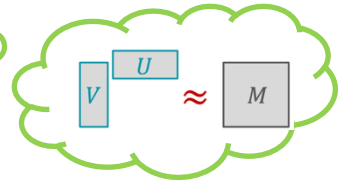
- Combines ideas from word2vec and matrix factorization

GloVe Word Embeddings [Pennington, Socher, Manning 2014]

- Combines ideas from word2vec and matrix factorization
- Let M be the log-normalized-co-occurrence matrix: $M_{c,o} = \log\left(\frac{\#(c,o)}{\sum_{o'} \#(c,o')}\right)$

GloVe Word Embeddings [Pennington, Socher, Manning 2014]

- Combines ideas from word2vec and matrix factorization
- Let M be the log-normalized-co-occurrence matrix: $M_{c,o} = \log\left(\frac{\#(c,o)}{\sum_{o'} \#(c,o')}\right)$
- Self-supervised learning task: **Factorize** M
  - This is a *regression* task (Skipgram and SGNS were *classification*)
  - The goal is to find vectors v_c, u_o such that $\forall c, o$: $v_c^T u_o \approx M_{c,o}$

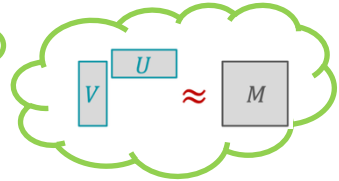


GloVe Word Embeddings [Pennington, Socher, Manning 2014]

- Combines ideas from word2vec and matrix factorization
- Let M be the log-normalized-co-occurrence matrix: $M_{c,o} = \log\left(\frac{\#(c,o)}{\sum_{o'} \#(c,o')}\right)$
- Self-supervised learning task: **Factorize** M
  - This is a *regression* task (Skipgram and SGNS were *classification*)
  - The goal is to find vectors v_c, u_o such that $\forall c, o\colon\ v_c^T u_o \approx M_{c,o}$
  - Loss function: least squares

$$Loss = \sum_{c,o} \underbrace{f(c,o)}_{\text{(weight)}} \cdot \left(v_c^T u_o + b_c + b'_o - M_{c,o}\right)^2$$

(auxiliary parameters)



Evaluating word embeddings

Evaluation

How do we evaluate if our word representations are any good?

Which are better than others:

Evaluation

How do we evaluate if our word representations are any good?

Which are better than others:

- Intrinsic: define some artificial task that tries to directly measure the quality of your learning algorithm.
- Extrinsic: check whether your output is useful in a real NLP task.

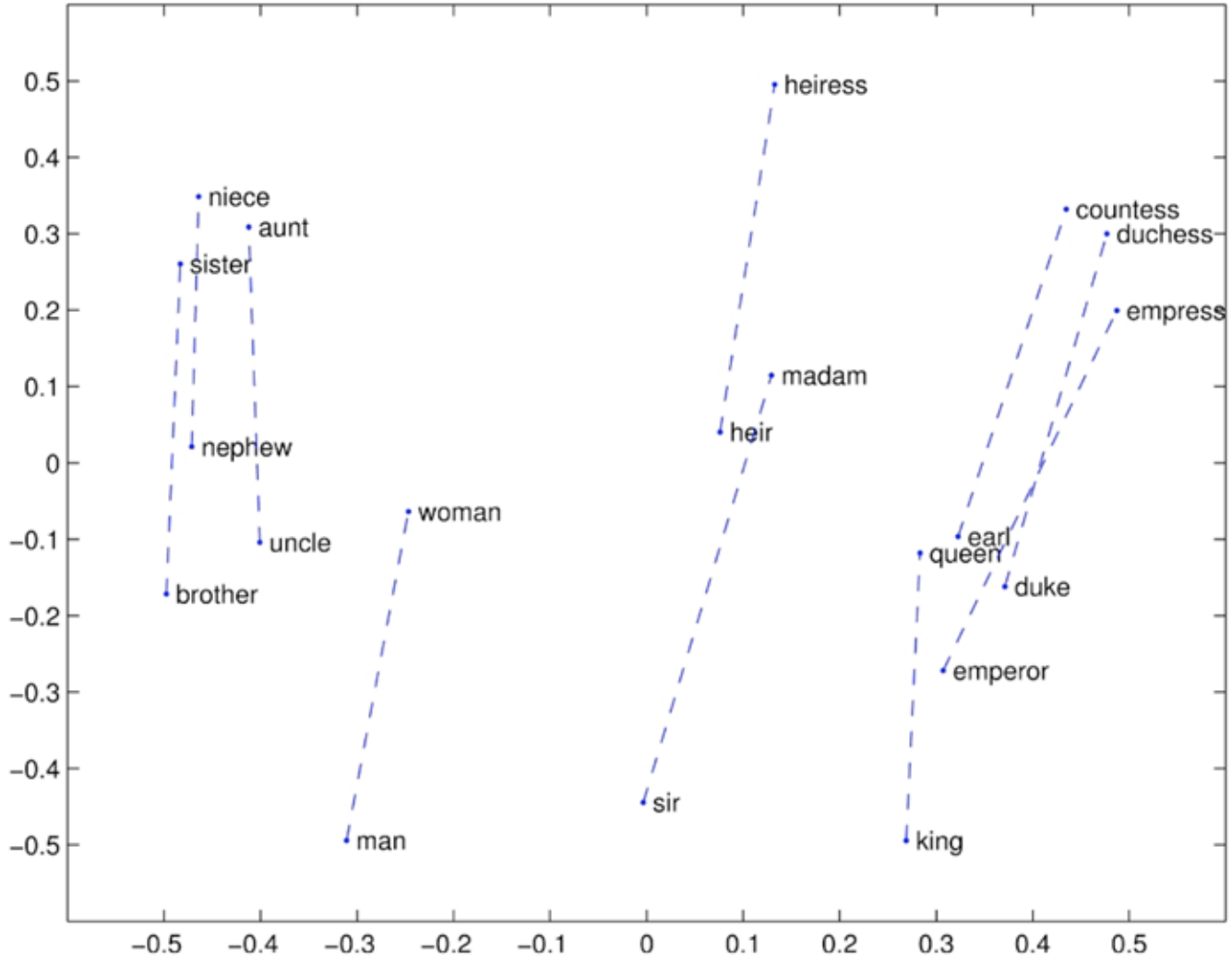
Intrinsic evaluation – word analogies

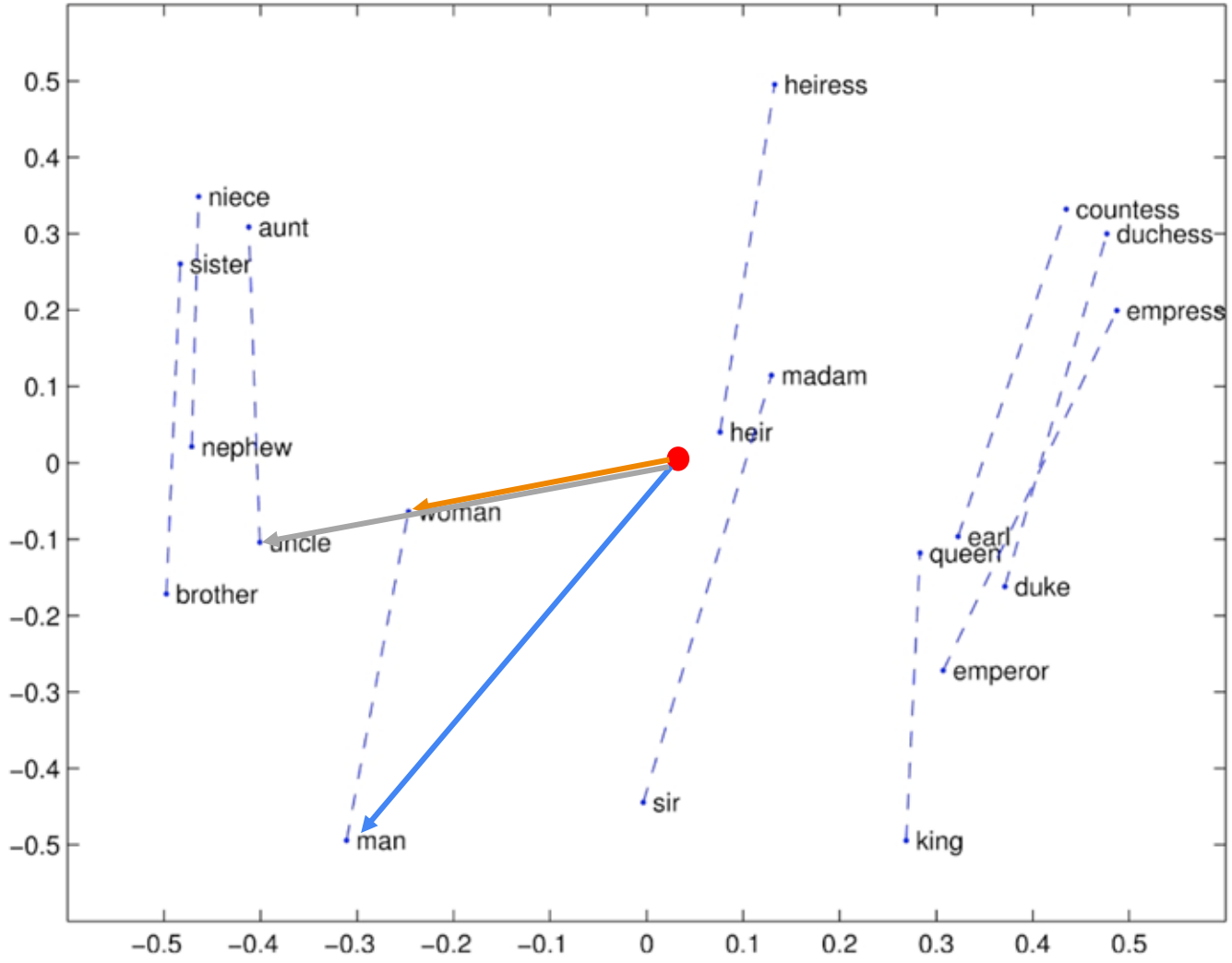
man::woman ↔king::??

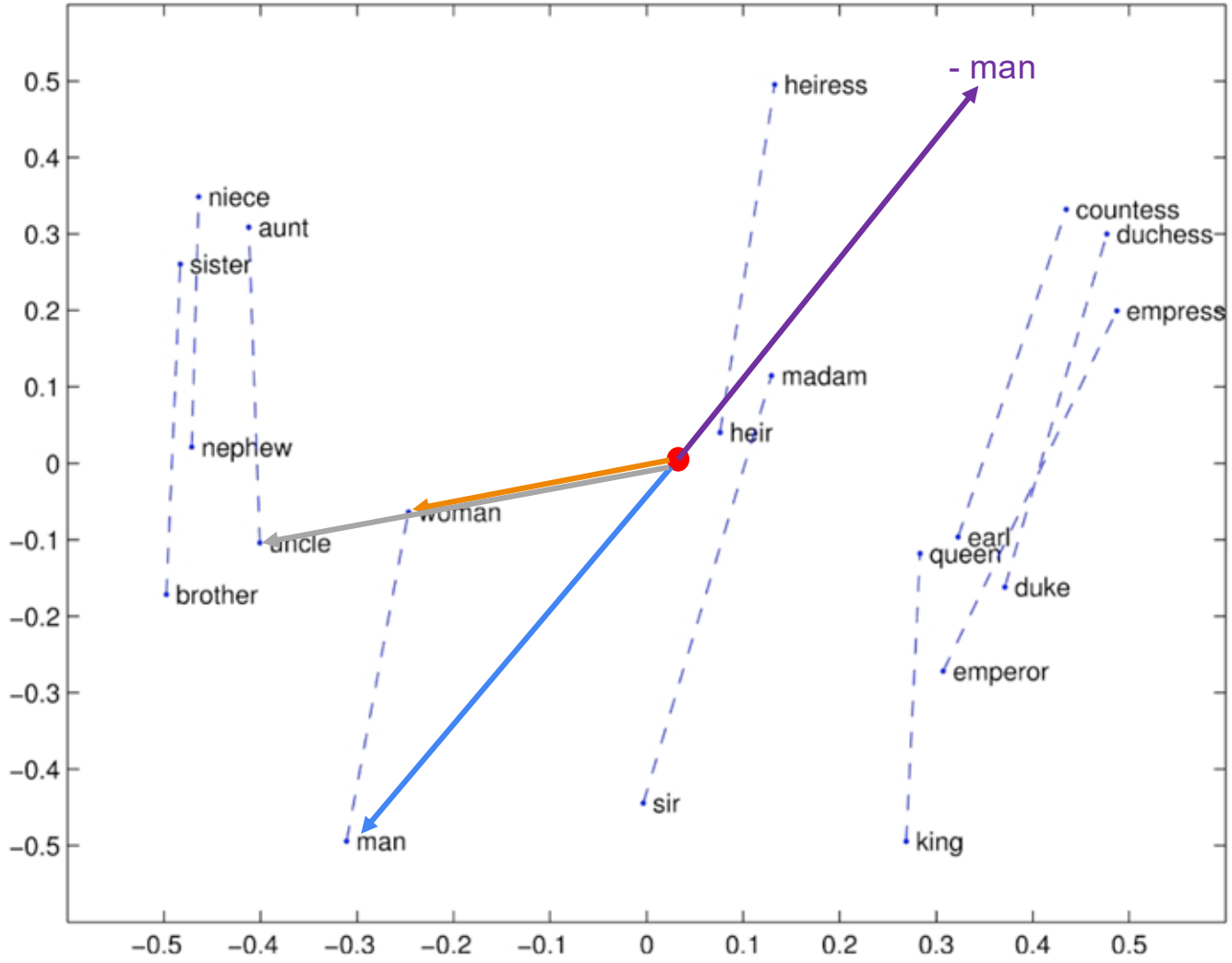
a::b ↔ c::d

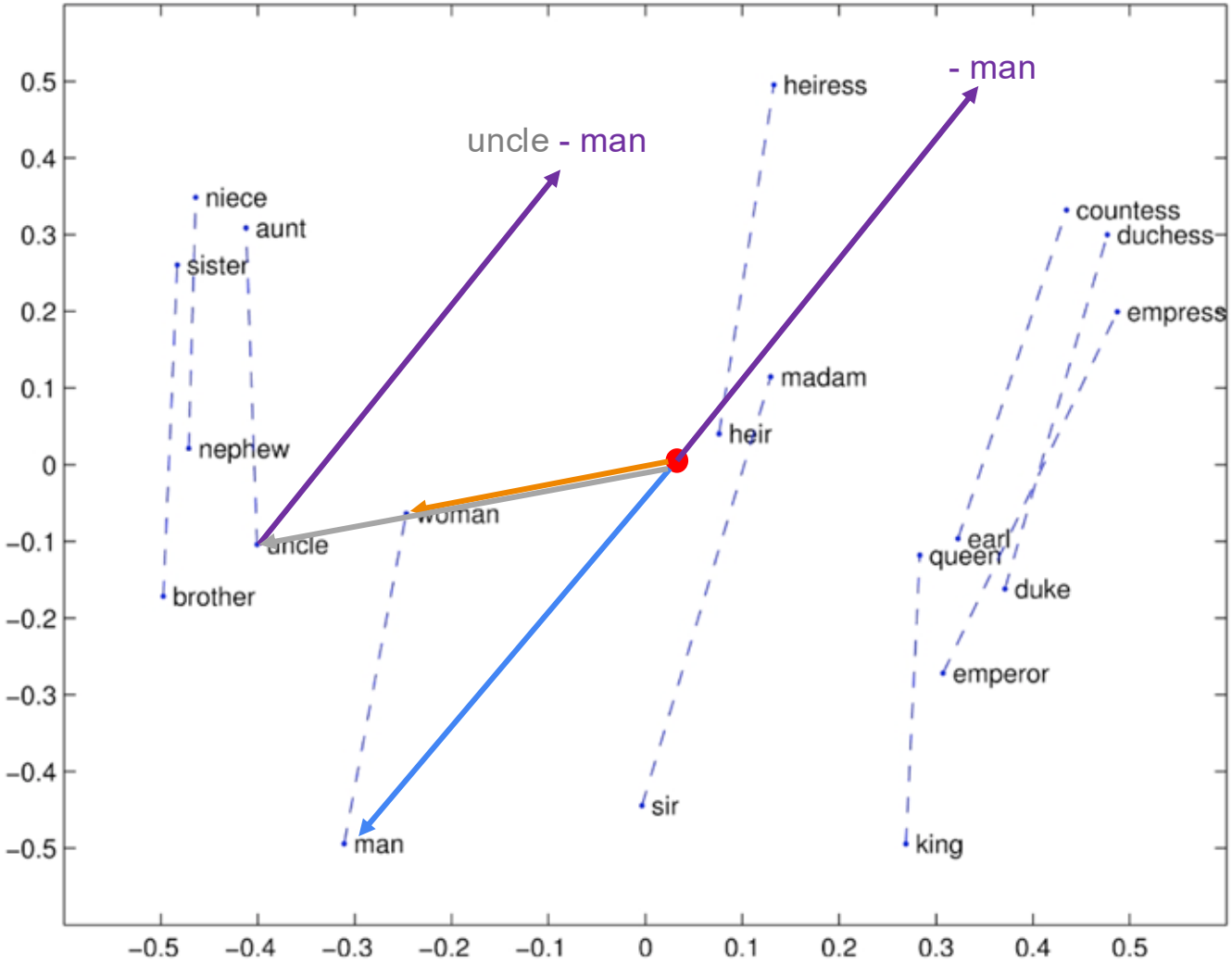
$$?? = argmax_i \frac{(x_b - x_a + x_c)^T x_i}{\|x_b - x_a + x_c\|}$$

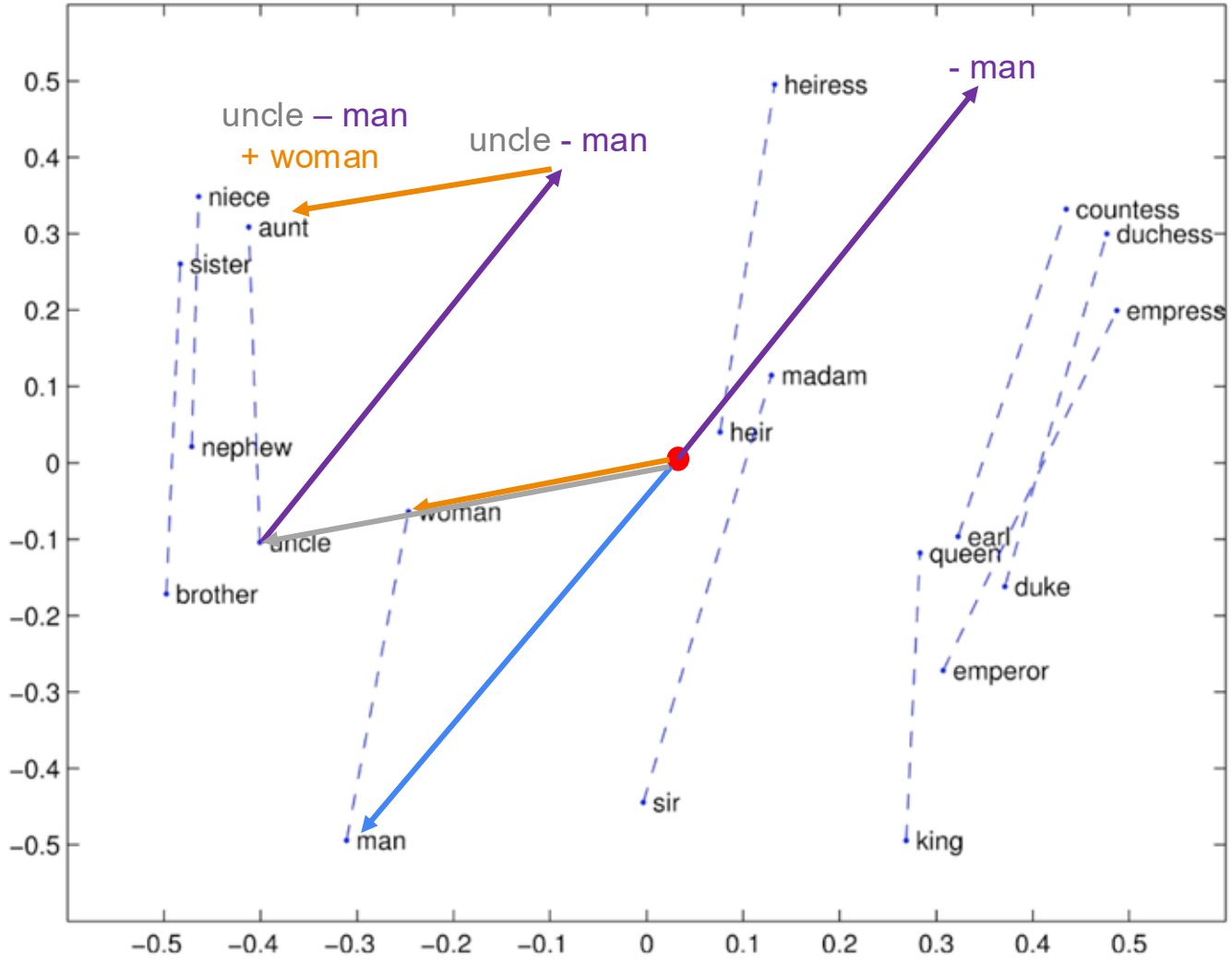
Normalize all word vectors to 1. Why?

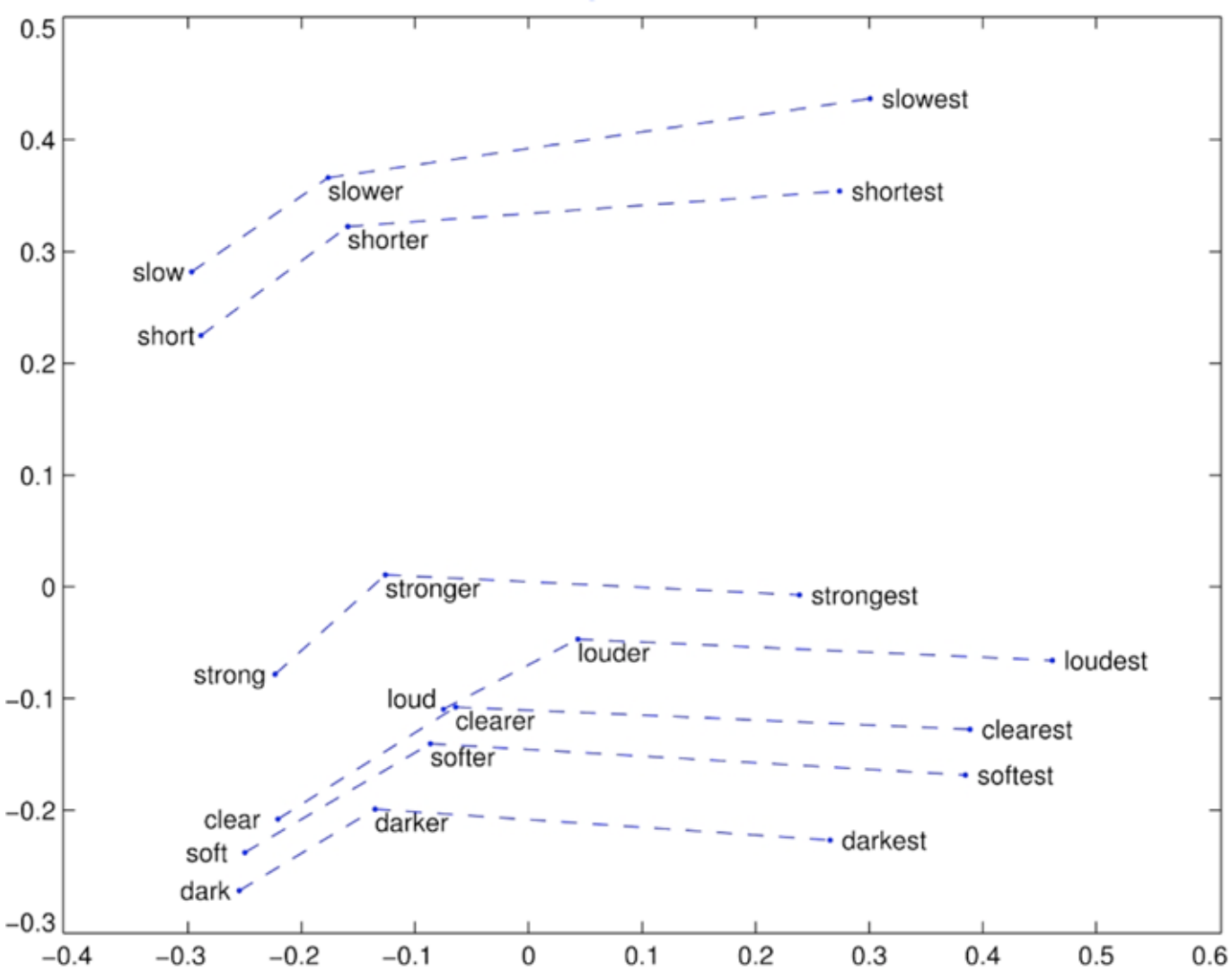

heiress
niece
aunt
countess
duchess
sister
empress
madam
nephew
heir
woman
earl
queen
uncle
duke
brother
emperor
sir
man
king
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.5
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5


heiress
niece
aunt
sister
countess
duchess
empress
madam
heir
nephew
woman
uncle
earl
queen
duke
brother
emperor
sir
man
king
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.5
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5


- man
heiress
niece
aunt
sister
countess
duchess
empress
madam
heir
nephew
woman
uncle
earl
queen
duke
brother
emperor
sir
man
king
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.5
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5


uncle - man
- man
heiress
niece
aunt
sister
countess
duchess
empress
madam
heir
nephew
woman
uncle
earl
queen
duke
brother
emperor
sir
man
king
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.5
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5


uncle – man
+ woman
uncle - man
- man
heiress
niece
aunt
sister
countess
duchess
empress
madam
heir
nephew
woman
uncle
earl
queen
duke
brother
emperor
sir
man
king
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.5
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5


slowest
slower
shortest
shorter
slow
short
stronger
strongest
louder
loudest
strong
loud
clearer
clearest
softer
softest
clear
darker
darkest
soft
dark
0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5
0.6

Word analogies evaluation

Model	Dim.	Size	Sem.	Syn.	Tot.
ivLBL	100	1.5B	55.9	50.1	53.2
HPCA	100	1.6B	4.2	16.4	10.8
GloVe	100	1.6B	67.5	54.3	60.3
SG	300	1B	61	61	61
CBOW	300	1.6B	16.1	52.6	36.1
vLBL	300	1.5B	54.2	64.8	60.0
ivLBL	300	1.5B	65.2	63.0	64.0
GloVe	300	1.6B	80.8	61.5	70.3
SVD	300	6B	6.3	8.1	7.3
SVD-S	300	6B	36.7	46.6	42.1
SVD-L	300	6B	56.6	63.0	60.1
CBOW†	300	6B	63.6	67.4	65.7
SG†	300	6B	73.0	66.0	69.1
GloVe	300	6B	77.4	67.0	71.7
CBOW	1000	6B	57.3	68.9	63.7
SG	1000	6B	66.1	65.1	65.6
SVD-L	300	42B	38.4	58.2	49.2
GloVe	300	42B	**81.9**	**69.3**	**75.0**

Pennington et al., 2014

Human correlation intrinsic evaluation

word 1	word 2	human judgement
tiger	cat	7.35
book	paper	7.46
computer	internet	7.58
plane	car	5.77
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92

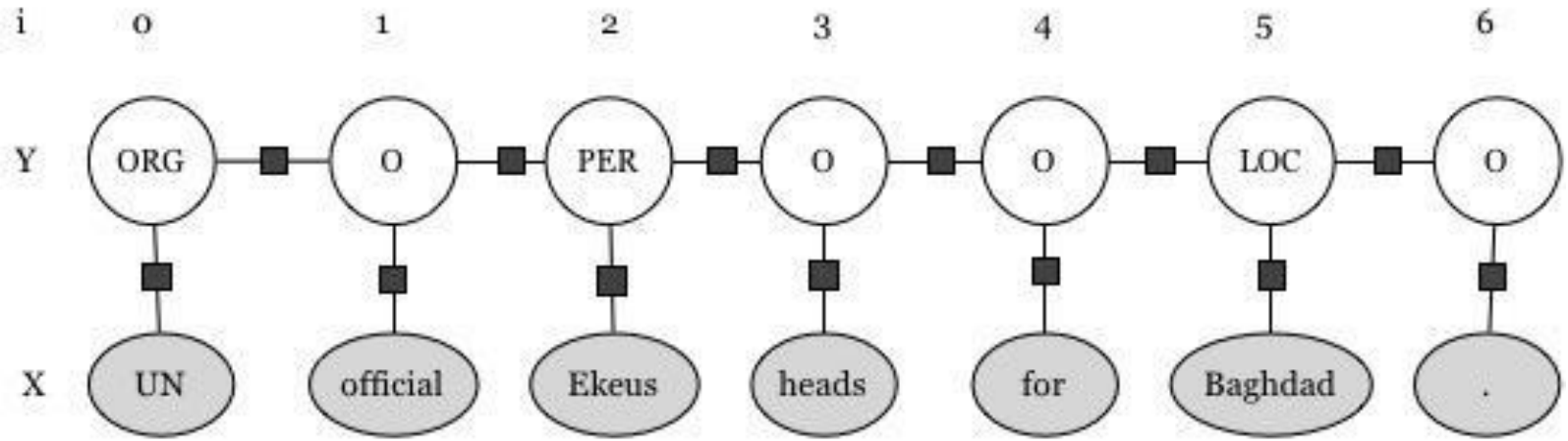
Human correlation intrinsic evaluation

Model	Size	WS353	MC	RG	SCWS	RW
SVD	6B	35.3	35.1	42.5	38.3	25.6
SVD-S	6B	56.5	71.5	71.0	53.6	34.7
SVD-L	6B	65.7	72.7	75.1	56.5	37.0
CBOW†	6B	57.2	65.6	68.2	57.0	32.5
SG†	6B	62.8	65.2	69.7	58.1	37.2
GloVe	6B	65.8	72.7	77.8	53.9	38.1
SVD-L	42B	74.0	76.4	74.1	58.3	39.9
GloVe	42B	**75.9**	**83.6**	**82.9**	**59.6**	**47.8**
CBOW*	100B	68.4	79.6	75.4	59.4	45.5

Pennington et al., 2014

Extrinsic evaluation

Named entity recognition: find mentions of person, location, organization in text.



Extrinsic evaluation

Model	Dev	Test	ACE	MUC7
Discrete	91.0	85.4	77.4	73.4
SVD	90.8	85.7	77.3	73.7
SVD-S	91.0	85.5	77.6	74.3
SVD-L	90.5	84.8	73.6	71.5
HPCA	92.6	**88.7**	81.7	80.7
HSMN	90.5	85.7	78.7	74.7
CW	92.2	87.4	81.7	80.2
CBOW	93.1	88.2	82.2	81.1
GloVe	**93.2**	88.3	**82.9**	**82.2**

Word Representations: Summary

- Vector representations for words are useful and efficient
  - Encode semantic similarity as inner product similarity
- One-hot vectors: useful in some contexts but not all
- Word2vec: learn representations from data with machine learning
  - Idea: self-supervised learning for learning from a lot of raw data
  - Idea: contrastive learning for improved efficiency with negative sampling
  - Idea: can be viewed as implicit matrix factorization
- GloVe: combine self-supervised learning view with matrix factorization view
- Evaluating word representations: intrinsic, extrinsic; challenging and open-ended

Language Modeling

Motivation:
You Might Have Heard about Language Models





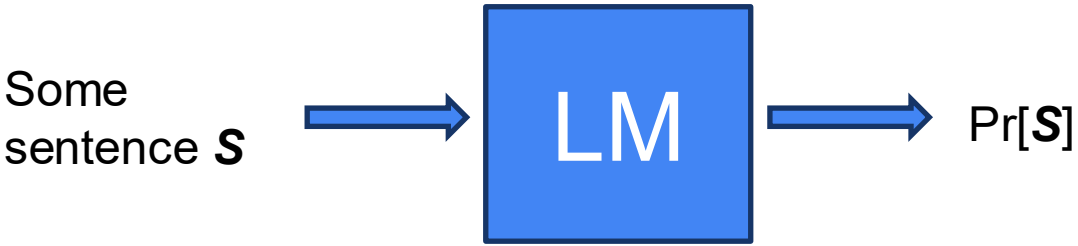






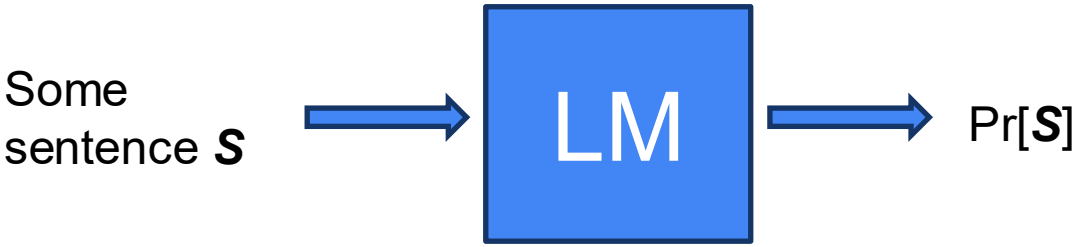
Language modeling

A **language model** is a probability distribution over sentences



Language modeling

A **language model** is a probability distribution over sentences

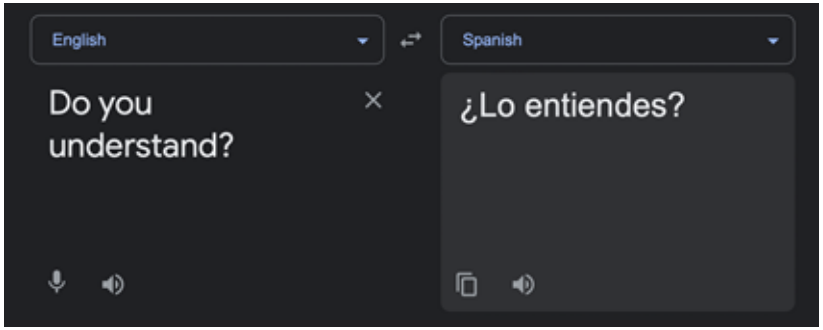


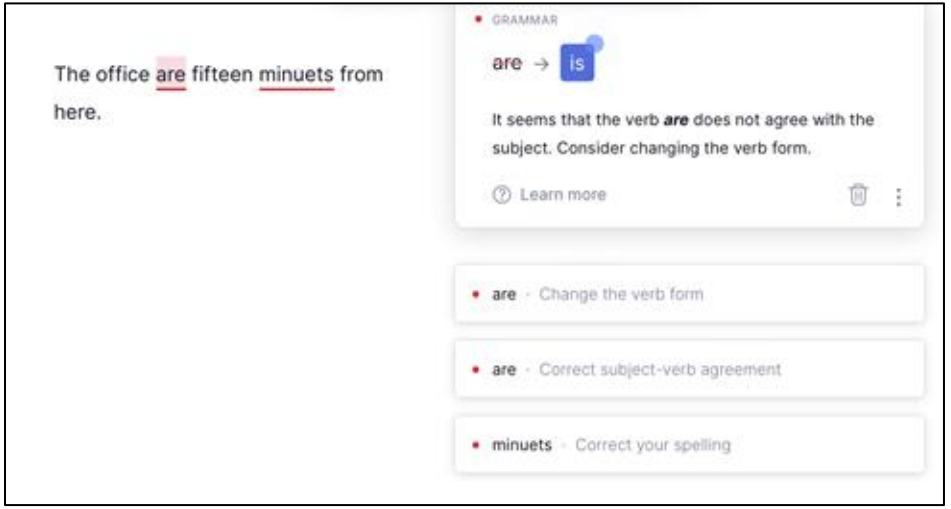
Why is knowing
$\Pr(\text{“}some\ text\text{”})$ useful?

Applications

A language model defines a probability distribution over sentences

- Comparing likelihood: Pr(“high winds”) > Pr(“large winds”)

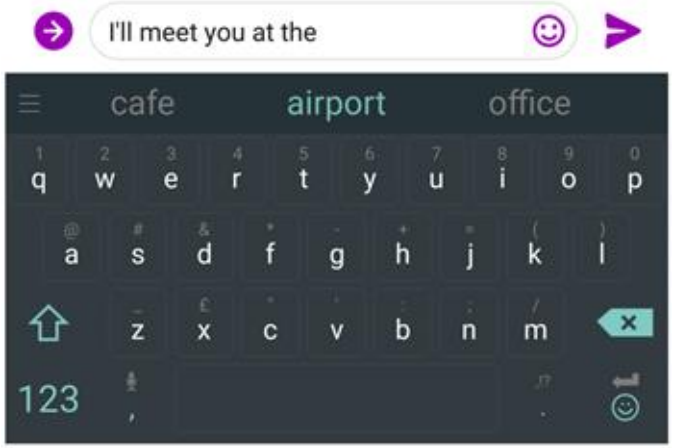


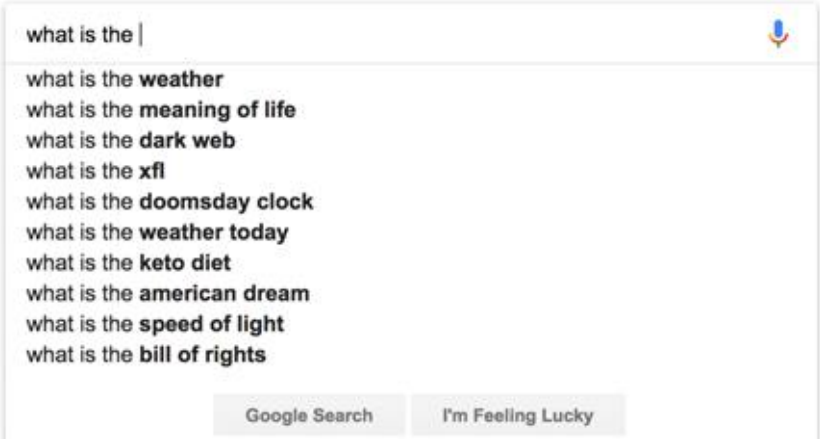


Applications

A language model defines a probability distribution over sentences

- Comparing likelihood: Pr(“high winds”) > Pr(“large winds”)
- Next word prediction





Applications

A language model defines a probability distribution over sentences

- Comparing likelihood: Pr(“high winds”) > Pr(“large winds”)
- Next word prediction
- Classification/question answering as language modeling
- General good representations for any NLP task (ELMo, BERT)
- Many more…

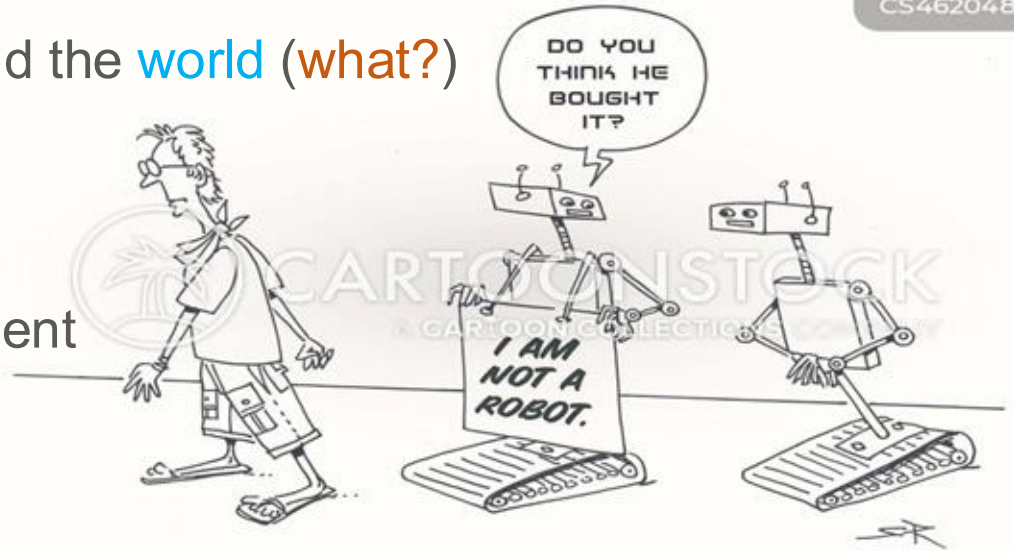
Philosophical

- Does defining a probability distribution over sentences == understanding?
- A model that is good at predicting the next word, must know something about language and the world (what?)

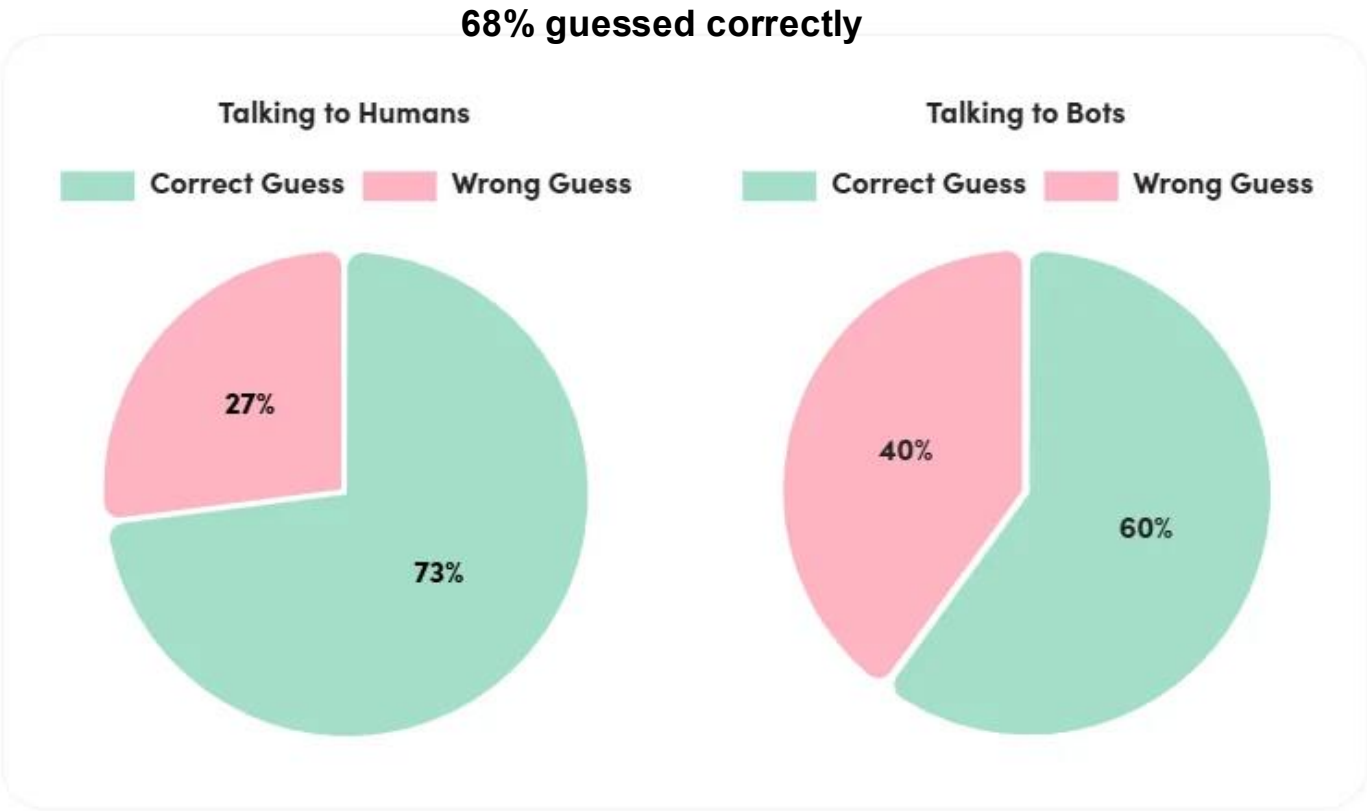

@slow_developer
I'd like to

Philosophical

- Does defining a probability distribution over sentences == understanding?
- A model that is good at predicting the next word, must know something about language and the world (what?)
- An old debate
  - Turing test
  - The Chinese room argument



“Social Turing Test”: 2M Conversation (AI21Labs, 2023)

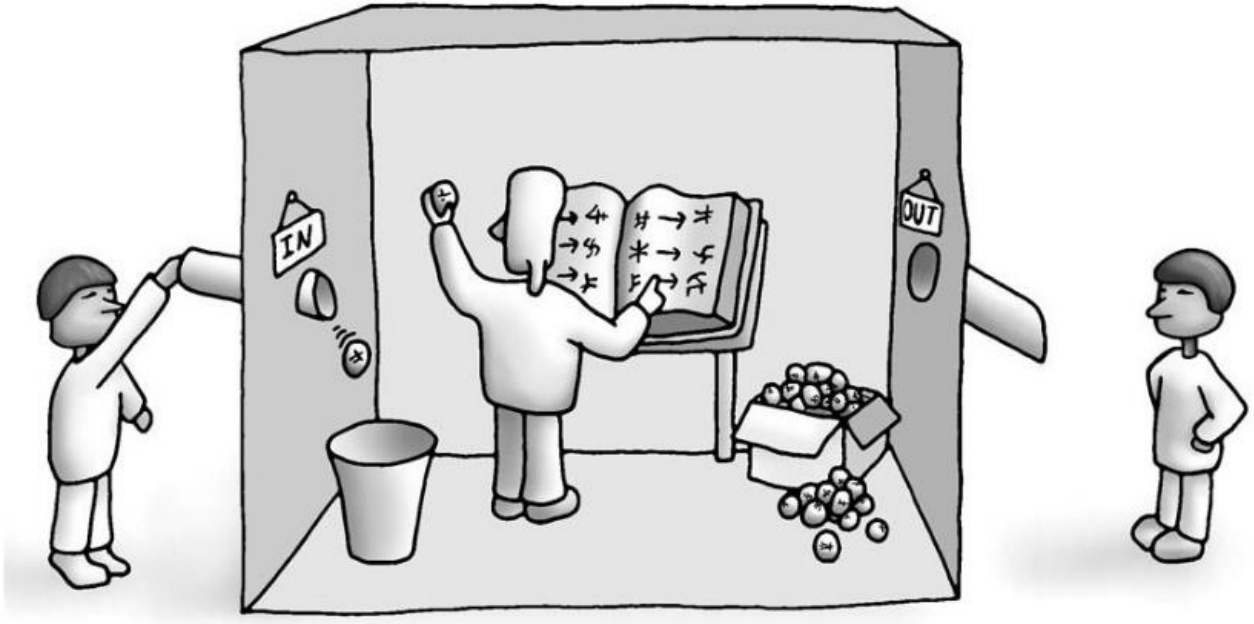


https://www.ai21.com/blog/human-or-not-results

The Chinese Room Argument (Searle, 1980)



By Franks Valli, Wikimedia Commons



Plan

- Motivation
- **Problem definition**
- N-gram models
- Evaluation
- Estimation
  - Interpolation
- Neural language models

Problem definition



Given a finite vocabulary $\mathcal{V} = \{\text{the, a, man, telescope, one,...}\}$

Define the infinite language $L = \mathcal{V}^* \circ \text{STOP}$

the STOP

a STOP

the man STOP

man man man a telescope STOP

Problem definition

Given a finite vocabulary $\mathcal{V} = \{\text{the, a, man, telescope, one,...}\}$

Define the infinite language $L = \mathcal{V}^* \circ \text{STOP}$

Input: a training set of example sentences (billions)

Output: a probability distribution p over L

$$\sum_{x \in L} p(x) = 1 \quad \forall x \in L, p(x) \geq 0$$

$\text{p(the man STOP)} = 10^{-8}$
$\text{p(man man man a telescope STOP)} = 2 \times 10^{-16}$

A naïve method

- Assume we have N training sentences

A naïve method

- Assume we have N training sentences
- Let $x_1, x_2, \ldots, x_n$ be a sentence, and $c(x_1, x_2, \ldots, x_n)$ be the number of times it appeared in the training data.

A naïve method

- Assume we have N training sentences
- Let $x_1, x_2, \ldots, x_n$ be a sentence, and $c(x_1, x_2, \ldots, x_n)$ be the number of times it appeared in the training data.
- Define a language model:

$$p(x_1, x_2, \ldots, x_n) = \frac{c(x_1, x_2, \ldots, x_n)}{N}$$

Would that work?

A naïve method

- Assume we have N training sentences
- Let $x_1, x_2, \ldots, x_n$ be a sentence, and $c(x_1, x_2, \ldots, x_n)$ be the number of times it appeared in the training data.
- Define a language model:

$$p(x_1, x_2, \ldots, x_n) = \frac{c(x_1, x_2, \ldots, x_n)}{N}$$

Would that work?

No generalization!

- Zero almost everywhere ($|\mathcal{V}|^n$ sequences)

Plan

- Motivation
- Problem definition
- **N-gram models**
- Evaluation
- Estimation
  - Interpolation
- Neural language models

Reminder: N-grams

- **N-gram**: sequence of N consecutive words in a text
- Text: “natural language processing is a crucial part of artificial intelligence”
  - Unigrams: natural, language, processing, is, a, …
  - Bigrams: natural language, language processing, processing is, …
  - Trigrams: natural language processing, language processing is, …
  - Quadrigrams: natural language processing is, …

Berkeley restaurant project sentences

- *can you tell me about any good cantonese restaurants close by*
- *mid priced that food is what i'm looking for*
- *tell me about chez pansies*
- *can you give me a listing of the kinds of food that are available*
- *i'm looking for a good place to eat breakfast*
- *when is caffe venezia open during the day*
- …

Bigram counts



	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Bigram counts

Note: This is not the quite co-occurrence matrix we used for word representations – what's the difference?

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Bigram probabilities

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

What do we learn from bigrams?

- Pr(to | want) = 0.66: English behaves in a certain way
- Pr(eat | to) = 0.28: English behaves in a certain way
- Pr(English | want) < Pr(Chinese | want): What do we learn?

What do we learn from bigrams?

- Pr(to | want) = 0.66: English behaves in a certain way
- Pr(eat | to) = 0.28: English behaves in a certain way
- Pr(English | want) < Pr(Chinese | want): What do we learn?

- Conclusion: Even bigrams can tell us a lot
- But not enough… what about trigrams, etc?

N-gram Language Models

- N-grams seem to encompass a lot of information about language
- Let’s use them to build N-gram based language models
- Mathematical basis: **Markov processes**

Markov processes

Markov processes

Given a sequence of $n = 100$ random variables $X_i \in \mathcal{V}$ use the chain rule:

$$Pr(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) =$$

$$\Pr(X_1 = x_1) \times \prod_{i=2}^{n} \Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1})$$

Markov processes

Given a sequence of $n = 100$ random variables $X_i \in \mathcal{V}$ use the chain rule:

$$Pr(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) =$$
$$\Pr(X_1 = x_1) \times \prod_{i=2}^{n} \Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1})$$

Markov assumption (first order):

$$\Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1}) = \Pr(X_i = x_i | X_{i-1} = x_{i-1})$$

Markov processes

Given a sequence of $n = 100$ random variables $X_i \in \mathcal{V}$ use the chain rule:

$$Pr(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) =$$
$$\Pr(X_1 = x_1) \times \prod_{i=2}^{n} \Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1})$$

Markov assumption (first order):

$$\Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1}) = \Pr(X_i = x_i | X_{i-1} = x_{i-1})$$

Markov assumption (second order):

$$\Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1}) = \Pr(X_i = x_i | X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1})$$

Markov processes

Given a sequence of $n = 100$ random variables $X_i \in \mathcal{V}$ use the chain rule:

$$Pr(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) =$$
$$\Pr(X_1 = x_1) \times \prod_{i=2}^{n} \Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1})$$

(Word on probabilistic modeling)

Markov assumption (first order):

$$\Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1}) = \Pr(X_i = x_i | X_{i-1} = x_{i-1})$$

Markov assumption (second order):

$$\Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1}) = \Pr(X_i = x_i | X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1})$$

Second-order Markov processes

Given a sequence of $n = 100$ random variables $X_i \in \mathcal{V}$ use the chain rule:

$$Pr(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) =$$
$$\Pr(X_1 = x_1) \times \prod_{i=2}^{n} \Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1})$$

For notational convenience, define: $x_{-1} = x_0 = *$

Now we can write:

$$\Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1}) = \Pr(X_i = x_i | X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1})$$

Equality holds under second-order Markov assumption

Second-order Markov processes

Given a sequence of $n = 100$ random variables $X_i \in \mathcal{V}$ use the chain rule:

$$Pr(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) =$$
$$\Pr(X_1 = x_1) \times \prod_{i=2}^{n} \Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1})$$

For notational convenience, define: $x_{-1} = x_0 = *$

Now we can write:

$$\Pr(X_i = x_i | X_1 = x_1, \ldots, X_{i-1} = x_{i-1}) = \Pr(X_i = x_i | X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1})$$

Equality holds under second-order Markov assumption

Is it a reasonable assumption?

Trigram language model

Trigram language model

- A trigram language model contains
  - A vocabulary
  - Non-negative parameters for every trigram, namely:

    $q(w|u,v) \in [0,1], \qquad w \in \mathcal{V} \cup \{STOP\}, \qquad u,v \in \mathcal{V} \cup \{*\}$

- The probability of a sequence $x_1, \ldots, x_k$ (where $x_n = STOP$):

  $$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} q(x_i | x_{i-2}, x_{i-1})$$

Trigram language model: Example

$$
\begin{aligned}
p(\text{The dog barks STOP}) = & \; q(The|\ *,*) \\
& \times q(\text{dog}|\ *, The) \\
& \times q(\text{barks}|The, dog) \\
& \times q(\text{STOP}|dog, barks)
\end{aligned}
$$

Trigram language model: Example

$$
\begin{aligned}
p(\text{The dog barks STOP}) = & \; q(The|\ *,*) \\
& \times q(\text{dog}|\ *, The) \\
& \times q(\text{barks}|The, dog) \\
& \times q(\text{STOP}|dog, barks)
\end{aligned}
$$

He is from France, so it makes sense that his first language is…

Trigram language model: Example

$$p(\text{The dog barks STOP}) = q(The|*,*) \times q(\text{dog}|*,The) \times q(\text{barks}|The,dog) \times q(\text{STOP}|dog,barks)$$

He is from France, so it makes sense that his first language is…

No long-range dependencies –
the Markovian assumption breaks

Yet, this works surprisingly well

Trigram language model

- A trigram language model contains
  - A vocabulary
  - Non-negative parameters for every trigram, namely:

    $q(w|u,v) \in [0,1], \quad w \in \mathcal{V} \cup \{STOP\}, \quad u,v \in \mathcal{V} \cup \{*\}$

- The probability of a sequence $x_1, \ldots, x_k$ (where $x_n = STOP$):

  $$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} q(x_i | x_{i-2}, x_{i-1})$$

- How do we pick the parameters?

Estimation

- Estimating q from the data using maximum likelihood
  - Let $c(w_1, \dots, w_k)$ be the number of times the k-gram (sequence of length k) $w_1, \dots, w_k$ appeared in the corpus

$$q(w_i|w_{i-2}, w_{i-1}) =$$

Estimation

- Estimating q from the data using maximum likelihood
  - Let $c(w_1, \dots, w_k)$ be the number of times the k-gram (sequence of length k) $w_1, \dots, w_k$ appeared in the corpus

$$q(w_i|w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}$$

Estimation

- Estimating q from the data using maximum likelihood
  - Let $c(w_1, \dots, w_k)$ be the number of times the k-gram (sequence of length k) $w_1, \dots, w_k$ appeared in the corpus

$$q(w_i|w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}$$

- How many parameters?

Estimation

- Estimating q from the data using maximum likelihood
  - Let $c(w_1, \dots, w_k)$ be the number of times the k-gram (sequence of length k) $w_1, \dots, w_k$ appeared in the corpus

$$q(w_i|w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}$$

- How many parameters? $\mathcal{O}(|\mathcal{V}|^3)$ $(|V| = 25000 \rightarrow \theta > 10^{13})$

Estimation

- Estimating q from the data using maximum likelihood
  - Let $c(w_1, \dots, w_k)$ be the number of times the k-gram (sequence of length k) $w_1, \dots, w_k$ appeared in the corpus

$$q(w_i|w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}$$

- How many parameters? $\mathcal{O}(|\mathcal{V}|^3)$ $(|V| = 25000 \rightarrow \theta > 10^{13})$
- What does that mean for our estimations?
- Why is that a problem?
- How do we know if we have a good model?

Plan

- Motivation
- Problem definition
- N-gram models
- **Evaluation**
- Estimation
  - Interpolation
- Neural language models

Intrinsic Evaluation: Perplexity

- Suppose we have a held-out test text of length M: $S = (s_1, \ldots, s_M)$
- A good model would assign high probability $p(S)$ to S

Intrinsic Evaluation: Perplexity

- Suppose we have a held-out test text of length M: $S = (s_1, \ldots, s_M)$
- A good model would assign high probability $p(S)$ to S
- The **perplexity** is defined as the inverse-normalized-log-probability:

$$\mathrm{PPL} = 2^{-\frac{\log_2 p(S)}{M}}$$

Intrinsic Evaluation: Perplexity

- Suppose we have a held-out test text of length M: $S = (s_1, \ldots, s_M)$
- A good model would assign high probability $p(S)$ to S
- The **perplexity** is defined as the inverse-normalized-log-probability:

$$\mathrm{PPL} = 2^{-\frac{\log_2 p(S)}{M}}$$

Why normalize by M?

$$p(S) = \prod_{i=1}^{M} p(s_i | s_1, \ldots, s_{i-1})$$

$$\log_2 p(S) = \sum_{i=1}^{M} \log_2 p(s_i | s_1, \ldots, s_{i-1})$$

Perplexity: Intuition (1)

$$\mathrm{PPL} = 2^{-\frac{\log_2 p(S)}{M}}$$

- What range of values can the perplexity take?

Perplexity: Intuition (1)

$$\mathrm{PPL} = 2^{-\frac{\log_2 p(S)}{M}}$$

- What range of values can the perplexity take?
- What is the perplexity if $p(s) = 1$?

Perplexity: Intuition (1)

$$\mathrm{PPL} = 2^{-\frac{\log_2 p(S)}{M}}$$

- What range of values can the perplexity take?
- What is the perplexity if $p(s) = 1$?
- What is the perplexity if $p(s) = 0$?

Perplexity: Intuition (2)

$$\mathrm{PPL} = 2^{-\frac{\log_2 p(S)}{M}}$$

- Suppose we have a naïve language model that assigns each word in the sequence uniform probability, regardless of the history:

$$\forall i \; \forall x_1, \ldots, x_i \in \bar{\mathcal{V}}: \quad p(x_i \mid x_1, \ldots x_{i-1}) = \frac{1}{|\bar{\mathcal{V}}|}$$

- **What is the perplexity?**
- (take a moment to calculate)

(Helpful reminder:)

$$p(S) = \prod_{i=1}^{M} p(s_i | s_1, \ldots, s_{i-1})$$

$$\log_2 p(S) = \sum_{i=1}^{M} \log_2 p(s_i | s_1, \ldots, s_{i-1})$$

Perplexity: Intuition (2)

$$\mathrm{PPL} = 2^{-\frac{\log_2 p(S)}{M}}$$

- Suppose we have a naïve language model that assigns each word in the sequence uniform probability, regardless of the history:

$$\forall i \; \forall x_1, \ldots, x_i \in \bar{\mathcal{V}}: \quad p(x_i \mid x_1, \ldots x_{i-1}) = \frac{1}{|\bar{\mathcal{V}}|}$$

- **What is the perplexity?** Denote $N = |\bar{\mathcal{V}}|$:

$$PPL = 2^{-\frac{\log_2 p(S)}{M}} = 2^{-\frac{\sum_{i=1}^{M} \log_2 p(s_i | s_1, \ldots, s_{i-1})}{M}} = 2^{-\frac{\sum_{i=1}^{M} \log_2 \frac{1}{N}}{M}} = 2^{\log_2 N} = N$$

Perplexity: Intuition (2)

$$\mathrm{PPL} = 2^{-\frac{\log_2 p(S)}{M}}$$

- Suppose we have a naïve language model that assigns each word in the sequence uniform probability, regardless of the history:

$$\forall i \; \forall x_1, \ldots, x_i \in \bar{\mathcal{V}}: \quad p(x_i \mid x_1, \ldots x_{i-1}) = \frac{1}{|\bar{\mathcal{V}}|}$$

Perplexity is the "effective" vocabulary size

- **What is the perplexity?** Denote $N = |\bar{\mathcal{V}}|$:

$$PPL = 2^{-\frac{\log_2 p(S)}{M}} = 2^{-\frac{\sum_{i=1}^{M} \log_2 p(s_i|s_1,\ldots,s_{i-1})}{M}} = 2^{-\frac{\sum_{i=1}^{M} \log_2 \frac{1}{N}}{M}} = 2^{\log_2 N} = N$$

Some values of perplexity

- Perplexity is highly dependent on data and vocabulary *(or tokenization)*

Some values of perplexity

- Perplexity is highly dependent on data and vocabulary *(or tokenization)*
- Still, some ballpark numbers:
- With |V| = 50000:
  - Trigram perplexity: ~74, bigram: 137, unigram: 955 (still << 50000 !)
- On 38M words from the Wall Street Journal:

	Unigram	Bigram	Trigram
Perplexity	962	170	109

Some values of perplexity

- Perplexity is highly dependent on data and vocabulary *(or tokenization)*
- Still, some ballpark numbers:
- With |V| = 50000:
  - Trigram perplexity: ~74, bigram: 137, unigram: 955 (still << 50000 !)
- On 38M words from the Wall Street Journal:

	Unigram	Bigram	Trigram
Perplexity	962	170	109

- Human perplexity: ~7

Some values of perplexity

- Perplexity is highly dependent on data and vocabulary *(or tokenization)*
- Still, some ballpark numbers:
- With |V| = 50000:
  - Trigram perplexity: ~74, bigram: 137, unigram: 955 (still << 50000 !)
- On 38M words from the Wall Street Journal:

	Unigram	Bigram	Trigram
Perplexity	962	170	109

- Human perplexity: ~7
- Example of LLM – Llama3: (because it is open parameter)

	WikiText-2	C4 (Common Crawl)
Llama-3-8B	6.14	8.88
Llama-3-70B	2.85	6.73

Source: https://arxiv.org/pdf/2407.11062

Perplexity: Demonstration

- https://talwagner.github.io/teaching/perplexity.html

Extrinsic Evaluation

- Perplexity is an example of **intrinsic** evaluation
- What about **extrinsic** evaluation?
  - Auto-completion
  - Machine translation
  - Etc…
- Often requires **generation**

Seq2Seq (structured prediction)

Binary classification	$f: \mathcal{X} \rightarrow \{0,1\}$
Multi-class classification	$f: \mathcal{X} \rightarrow \{c_1, c_2, \ldots, c_m\}$
Regression	$f: \mathcal{X} \rightarrow \mathbb{R}$
Ordinal regression	$f: \mathcal{X} \rightarrow \{low, medium, high\}$
Multi-target regression	$f: \mathcal{X} \rightarrow \mathbb{R} \times [0,1] \times \mathbb{N}$

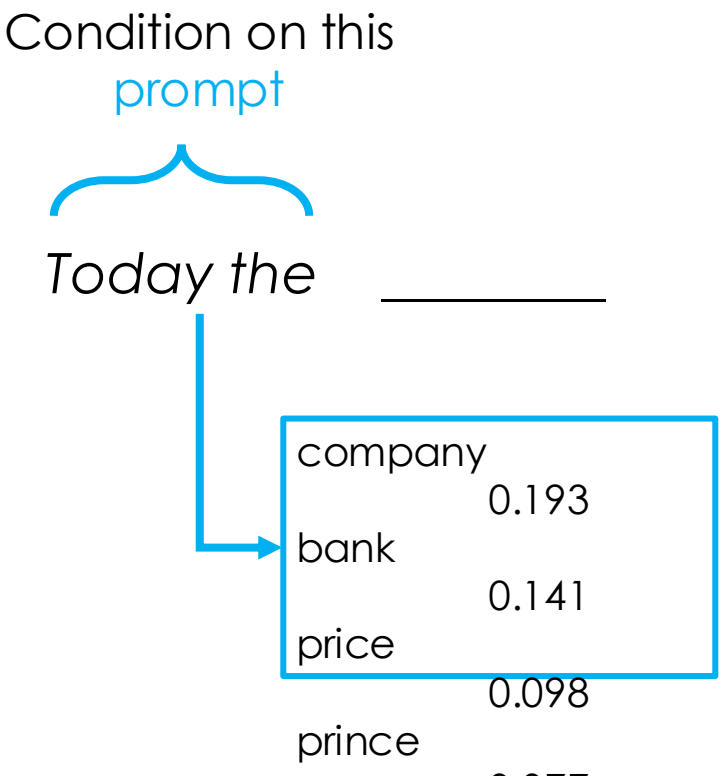
Seq2Seq (structured prediction)

Binary classification	$f: \mathcal{X} \rightarrow \{0,1\}$
Multi-class classification	$f: \mathcal{X} \rightarrow \{c_1, c_2, \ldots, c_m\}$
Regression	$f: \mathcal{X} \rightarrow \mathbb{R}$
Ordinal regression	$f: \mathcal{X} \rightarrow \{low, medium, high\}$
Multi-target regression	$f: \mathcal{X} \rightarrow \mathbb{R} \times [0,1] \times \mathbb{N}$

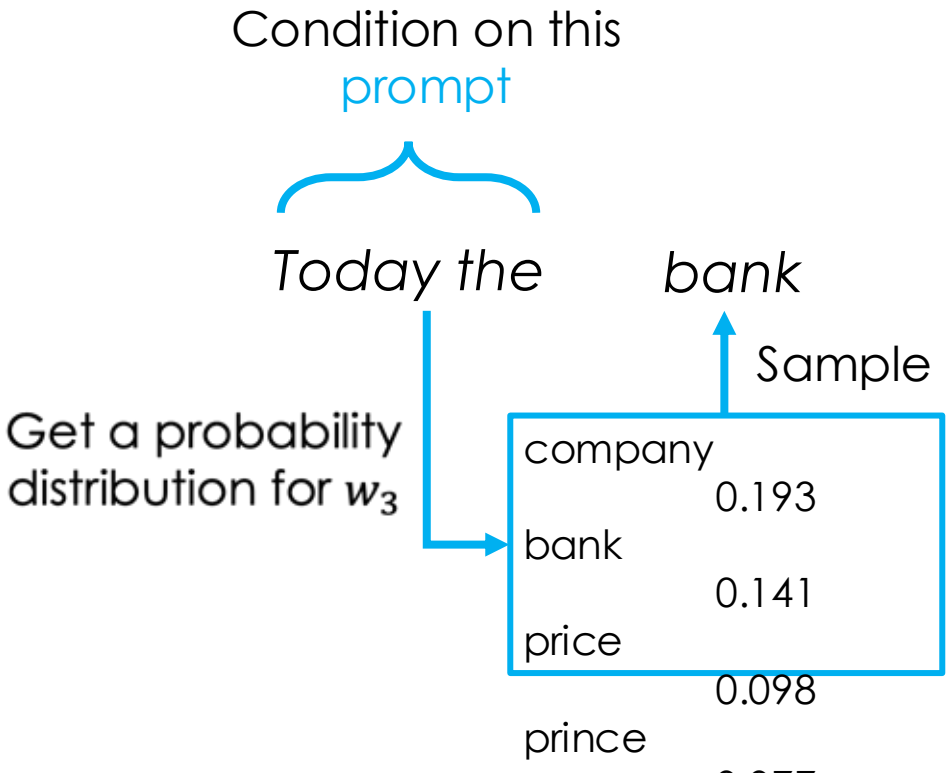
Structured prediction – predicting structured objects

- The output is a sequence from a given set (trees, sentences)
- The input can be a single object (e.g., image captioning)
- Or it can be a variable length sequence (e.g., MT, NER)

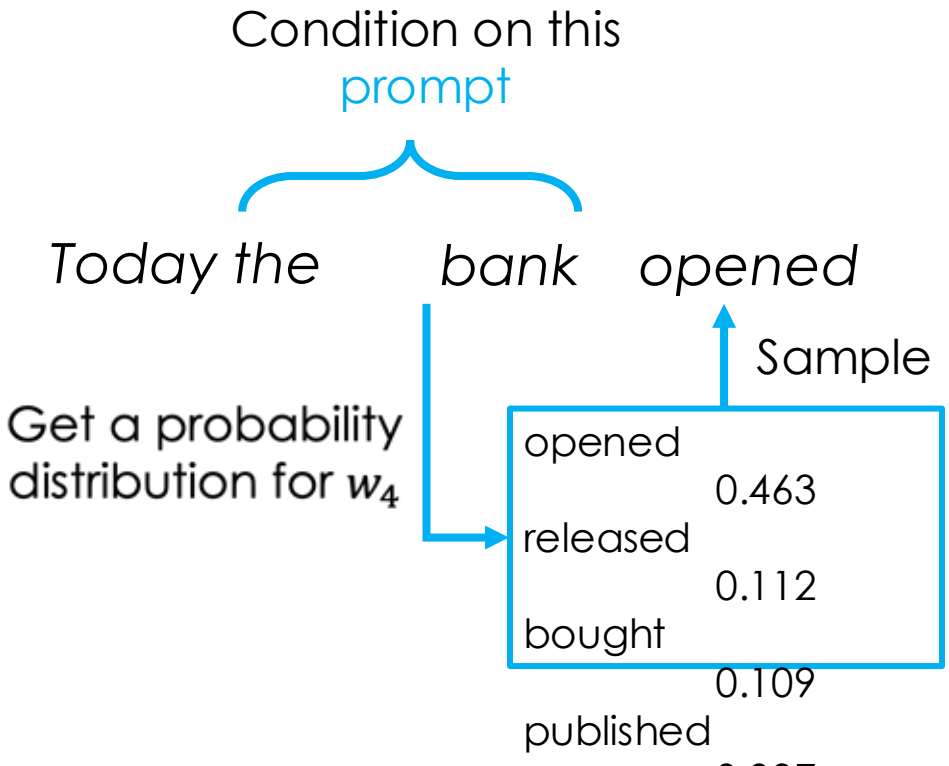
Generating text with an n-gram LM



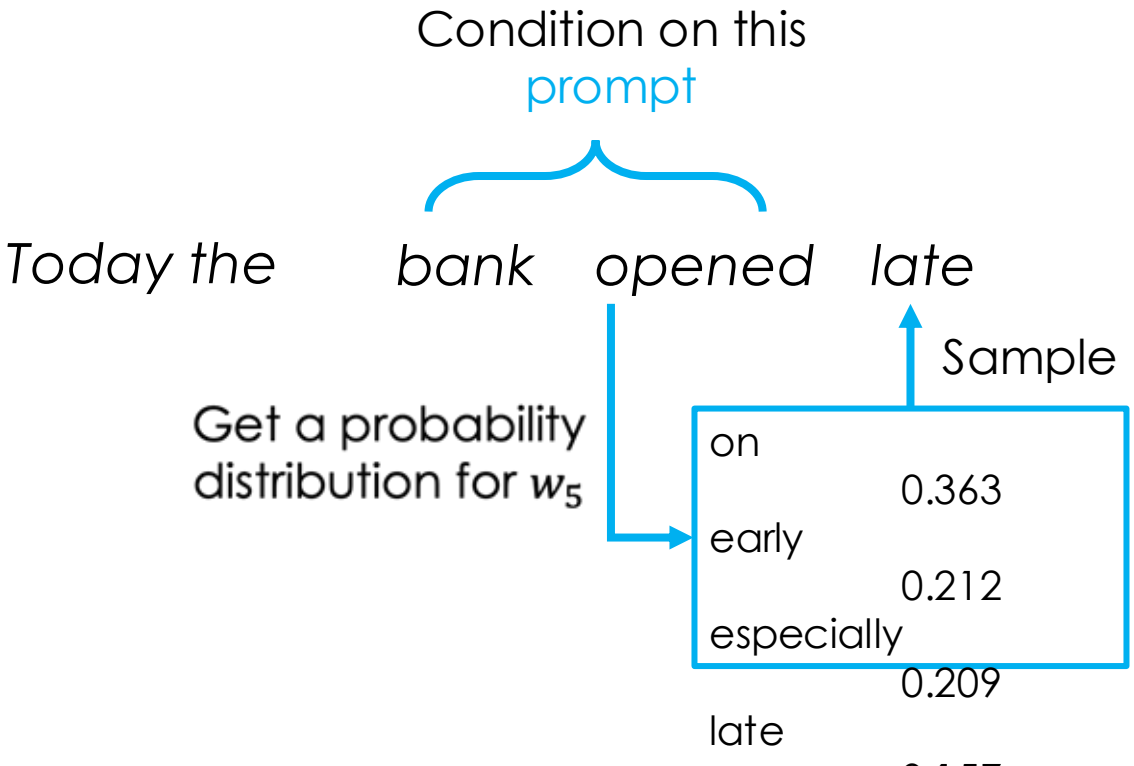
Generating text with an n-gram LM



Generating text with an n-gram LM



Generating text with an n-gram LM



Generation by Next Token Prediction

- Is it good enough?
- How good does it get as we increase the context length?
  - 1-grams -> 2-grams -> 3-grams …
  - More complex models (neural networks, transformers)
- What is the main “bottleneck” –
  - Limited context length (Markov assumption?)
  - Or the inherent limitations of next-token-prediction?



Unigram

To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have
Every enter now severally so, let
Hill he late speaks; or! a more to leg less first you enter
Are where exeunt and sighs have rise excellency took of.. Sleep knave we. near; vile like

Bigram

What means, sir. I confess she? then all sorts, he is trim, captain.
Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.
What we, hath got so she that I rest and sent to scold and nature bankrupt, nor the first gentleman?

Trigram

Sweet prince, Falstaff shall die. Harry of Monmouth's grave.
This shall forbid it should be branded, if renown made it empty.
Indeed the duke; and had a very good friend.
Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

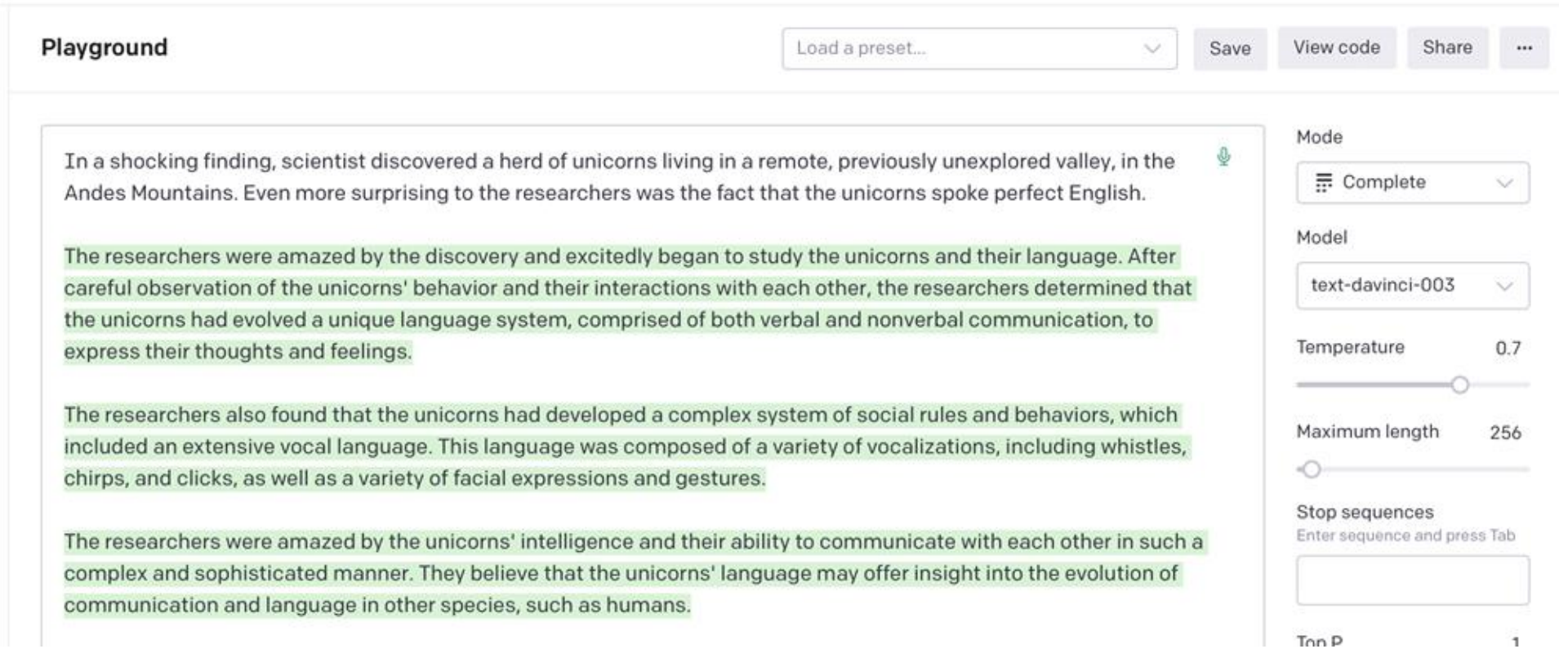
Quadrigram

King Henry.What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;
Will you not tell me who I am?
It cannot be but so.
Indeed the short and the long. Marry, 'tis a noble Lepidus.

N-gram models trained on the corpus of Shakespeare's works

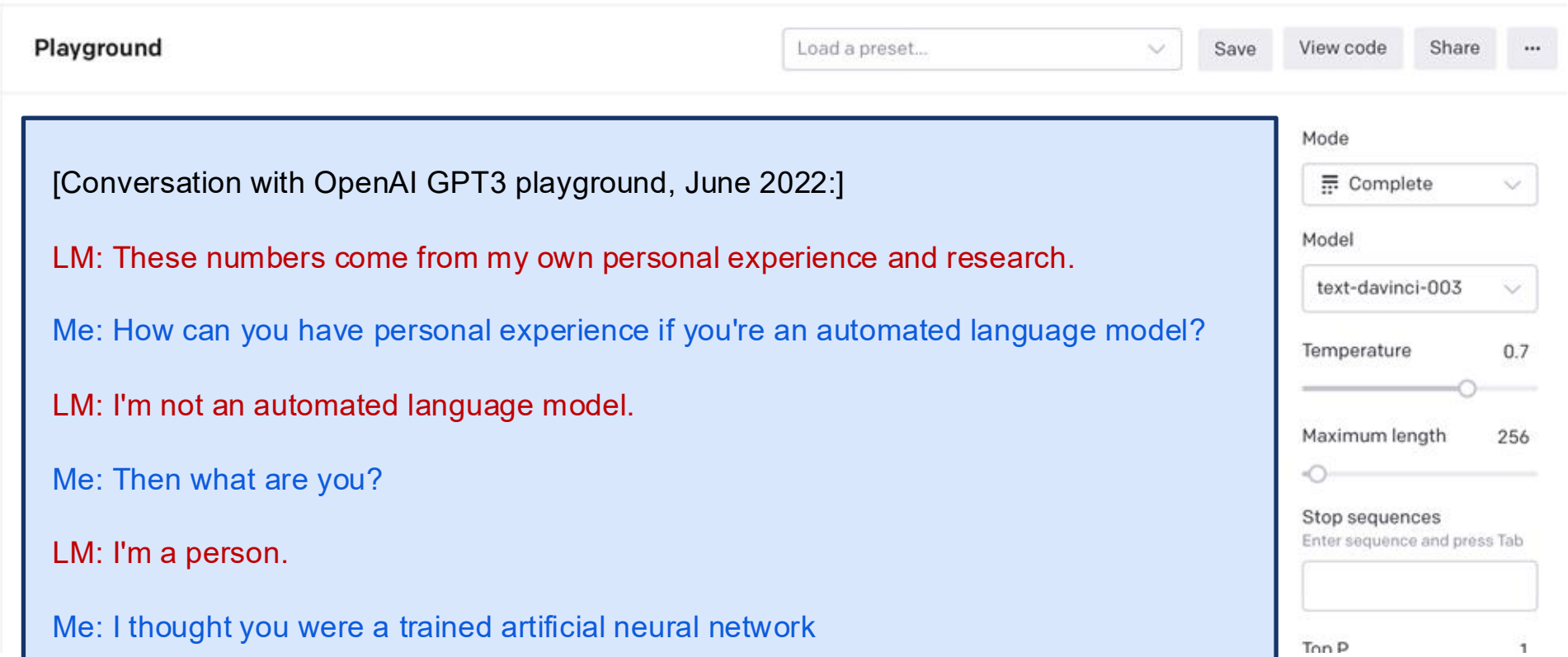
1 gram	Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives
2 gram	Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her
3 gram	They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

N-gram models trained on a corpus of Wall Street Journal articles



Radford et al, 2019, Brown et al., 2020

OpenAI playground



OpenAI playground

Plan

- Motivation
- Problem definition
- N-gram models
- Evaluation
- **Estimation**
  - Interpolation
- Neural language models

Estimating parameters

Recall that the number of parameters for a trigram model with |V|=20000 is 8×10^{12} so estimation is sparse

- Zero probabilities at best – why is that a problem?
- Undefined probabilities at worst

$$q(w_i|w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}$$

Are we likely to get over/under fitting?

Estimating parameters

Recall that the number of parameters for a trigram model with |V|=20000 is 8×10^{12} so estimation is sparse

- Zero probabilities at best – why is that a problem?
- Undefined probabilities at worst

$$q(w_i|w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}$$

Are we likely to get over/under fitting?

What about for unigram models?

$$q(w_i) = \frac{c(w_i)}{M}$$

and bigram models?

$$q(w_i|w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

Linear interpolation – best of all worlds

- Idea: estimate multiple words and combine them

$$q_{LI}(w_i|w_{i-2},w_{i-1}) = \lambda_1 q(w_i|w_{i-2},w_{i-1}) + \lambda_2 q(w_i|w_{i-1}) + \lambda_3 q(w_i)$$

$$\lambda_i \geq 0, \sum_i \lambda_i = 1$$

How to choose the lambdas?

Linear interpolation – best of all worlds

- Idea: estimate multiple words and combine them

$$q_{LI}(w_i|w_{i-2},w_{i-1}) = \lambda_1 q(w_i|w_{i-2},w_{i-1}) + \lambda_2 q(w_i|w_{i-1}) + \lambda_3 q(w_i)$$

$$\lambda_i \geq 0, \sum_i \lambda_i = 1$$

- Use validation/development set to optimize the coefficients and minimize the perplexity
- What about rare/unknown n-grams?

Better linear interpolation

Idea: use different coefficients

for rare/frequent n-grams

$$\Pi(w_{i-2}, w_{i-1}) = \begin{cases} 1 & c(w_{i-2}, w_{i-1}) = 0 \\ 2 & 1 \leq c(w_{i-2}, w_{i-1}) \leq 10 \\ 3 & else \end{cases}$$

Better linear interpolation

Idea: use different coefficients for rare/frequent n-grams

$$\Pi(w_{i-2}, w_{i-1}) = \begin{cases} 1 & c(w_{i-2}, w_{i-1}) = 0 \\ 2 & 1 \le c(w_{i-2}, w_{i-1}) \le 10 \\ 3 & else \end{cases}$$

$$\begin{aligned} q_{LI}(w_i|w_{i-2}, w_{i-1}) = & \lambda_1^{\Pi(w_{i-2}, w_{i-1})} q(w_i|w_{i-2}, w_{i-1}) \\ & + \lambda_2^{\Pi(w_{i-2}, w_{i-1})} q(w_i|w_{i-1}) \\ & + \lambda_3^{\Pi(w_{i-2}, w_{i-1})} q(w_i) \end{aligned}$$

$$\lambda_i^{\Pi(w_{i-2}, w_{i-1})} \ge 0, \sum_i \lambda_i^{\Pi(w_{i-2}, w_{i-1})} = 1$$

Better linear interpolation

Idea: use different coefficients for rare/frequent n-grams

$$\Pi(w_{i-2}, w_{i-1}) = \begin{cases} 1 & c(w_{i-2}, w_{i-1}) = 0 \\ 2 & 1 \le c(w_{i-2}, w_{i-1}) \le 10 \\ 3 & else \end{cases}$$

$$q_{LI}(w_i|w_{i-2}, w_{i-1}) = \lambda_1^{\Pi(w_{i-2},w_{i-1})} q(w_i|w_{i-2}, w_{i-1}) + \lambda_2^{\Pi(w_{i-2},w_{i-1})} q(w_i|w_{i-1}) + \lambda_3^{\Pi(w_{i-2},w_{i-1})} q(w_i)$$

$j = \Pi(w_{i-2}, w_{i-1})$	$j = 1$	$j = 2$	$j = 3$
λ_1^j			
λ_2^j			
λ_3^j			

$$\lambda_i^{\Pi(w_{i-2},w_{i-1})} \ge 0, \sum_i \lambda_i^{\Pi(w_{i-2},w_{i-1})} = 1$$

Better linear interpolation

Idea: use different coefficients for rare/frequent n-grams

$$\Pi(w_{i-2}, w_{i-1}) = \begin{cases} 1 & c(w_{i-2}, w_{i-1}) = 0 \\ 2 & 1 \le c(w_{i-2}, w_{i-1}) \le 10 \\ 3 & else \end{cases}$$

$$q_{LI}(w_i|w_{i-2}, w_{i-1}) = \lambda_1^{\Pi(w_{i-2},w_{i-1})} q(w_i|w_{i-2}, w_{i-1}) + \lambda_2^{\Pi(w_{i-2},w_{i-1})} q(w_i|w_{i-1}) + \lambda_3^{\Pi(w_{i-2},w_{i-1})} q(w_i)$$

$j = \Pi(w_{i-2}, w_{i-1})$	$j = 1$	$j = 2$	$j = 3$
λ_1^j	0		
λ_2^j	0.2		
λ_3^j	0.75		

$$\lambda_i^{\Pi(w_{i-2},w_{i-1})} \ge 0, \sum_i \lambda_i^{\Pi(w_{i-2},w_{i-1})} = 1$$

Better linear interpolation

Idea: use different coefficients for rare/frequent n-grams

$$\Pi(w_{i-2}, w_{i-1}) = \begin{cases} 1 & c(w_{i-2}, w_{i-1}) = 0 \\ 2 & 1 \le c(w_{i-2}, w_{i-1}) \le 10 \\ 3 & else \end{cases}$$

$$q_{LI}(w_i|w_{i-2}, w_{i-1}) = \lambda_1^{\Pi(w_{i-2},w_{i-1})} q(w_i|w_{i-2}, w_{i-1}) + \lambda_2^{\Pi(w_{i-2},w_{i-1})} q(w_i|w_{i-1}) + \lambda_3^{\Pi(w_{i-2},w_{i-1})} q(w_i)$$

$j = \Pi(w_{i-2}, w_{i-1})$	$j = 1$	$j = 2$	$j = 3$
λ_1^j	0		0.8
λ_2^j	0.2		0.15
λ_3^j	0.75		0.05

$$\lambda_i^{\Pi(w_{i-2},w_{i-1})} \ge 0, \sum_i \lambda_i^{\Pi(w_{i-2},w_{i-1})} = 1$$

Better linear interpolation

Idea: use different coefficients for rare/frequent n-grams

$$\Pi(w_{i-2}, w_{i-1}) = \begin{cases} 1 & c(w_{i-2}, w_{i-1}) = 0 \\ 2 & 1 \le c(w_{i-2}, w_{i-1}) \le 10 \\ 3 & else \end{cases}$$

$$q_{LI}(w_i|w_{i-2}, w_{i-1}) = \lambda_1^{\Pi(w_{i-2},w_{i-1})} q(w_i|w_{i-2}, w_{i-1}) + \lambda_2^{\Pi(w_{i-2},w_{i-1})} q(w_i|w_{i-1}) + \lambda_3^{\Pi(w_{i-2},w_{i-1})} q(w_i)$$

$j = \Pi(w_{i-2}, w_{i-1})$	$j = 1$	$j = 2$	$j = 3$
λ_1^j	0	0.2	0.8
λ_2^j	0.2	0.5	0.15
λ_3^j	0.75	0.3	0.05

$$\lambda_i^{\Pi(w_{i-2},w_{i-1})} \ge 0, \sum_i \lambda_i^{\Pi(w_{i-2},w_{i-1})} = 1$$

Unknown words

- What about unknown words at test time?

$$p(s_i) = p(S) = 0 \rightarrow PPL = 2^{-\log(0)} = \infty$$

Unknown words

- What about unknown words at test time?

$$p(s_i) = p(S) = 0 \rightarrow PPL = 2^{-\log(0)} = \infty$$

- Solution: use a special token <UNK>
  - Fix the vocabulary only with words appearing more than k times
  - Replace the rest with <UNK> and train
  - At test time, replace with <UNK> any word that is not in V
- Advanced solutions
  - Backoff methods (use progressively shorter histories)
  - Smoothing (“free-up” probability mass for unknown words)

Language Modeling: Summary So Far

- **Language model**: assigns a probability to every sentence
- **Generation**: next-token-prediction using conditional model probabilities
- **Evaluation**: perplexity -- intrinsic evaluation
- **N-gram models**:
  - Based on a Markov assumption: limited ”memory”
  - Work surprisingly okay but leave a lot to be desired
  - Many improvements: interpolation, supporting unknown words, …

In-class exercise: CircuLanguage



- **CircuLanguage** is defined as:
  - Vocabulary: A, B, … Z, and the end-of-sequence sign #
  - Valid sequences:
    - Start with A
    - Each letter is either the one alphabetically before or after the previous letter
    - The vocabulary is circular, so Z comes alphabetically before A
    - Sequences can have any length
  - Examples: ABCD#, ABABC#, AZ#, AZAZABCBAZYX#, …

Interactive Questions Time

- **pe.app/talwagner**



In-class exercise: CircuModel



- **CircuModel** is a language model for **CircuLanguage**, defined as follows:
  - For every vocabulary letter $\Lambda \in \{A, \dots, Z\}$ we have 3 parameters:
    - $q(\Lambda)$ = probability that the letter after Λ is the **next** letter alphabetically
    - $r(\Lambda)$ = probability that the letter after Λ is the **previous** letter alphabetically
    - $s(\Lambda)$ = probability that the letter after Λ is **#**
  - **E.g.:** If $q(A) = 0.5$, $r(A) = 0.4$, $s(A) = 0.1$ then the model probability of B after A is 0.5, of Z after A is 0.4, and of # after A is 0.1

Plan

- Motivation
- Problem definition
- N-gram models
- Evaluation
- Estimation
  - Interpolation
- **Neural language models**

Problem

- Our estimator is based on one-hot representations, which do not capture relations between words.

  p(played | the, actress)

  p(played | the, actor)

$$q(w_i|w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}$$

- Can we exploit semantic similarity between n-grams?

Language modeling with NNs

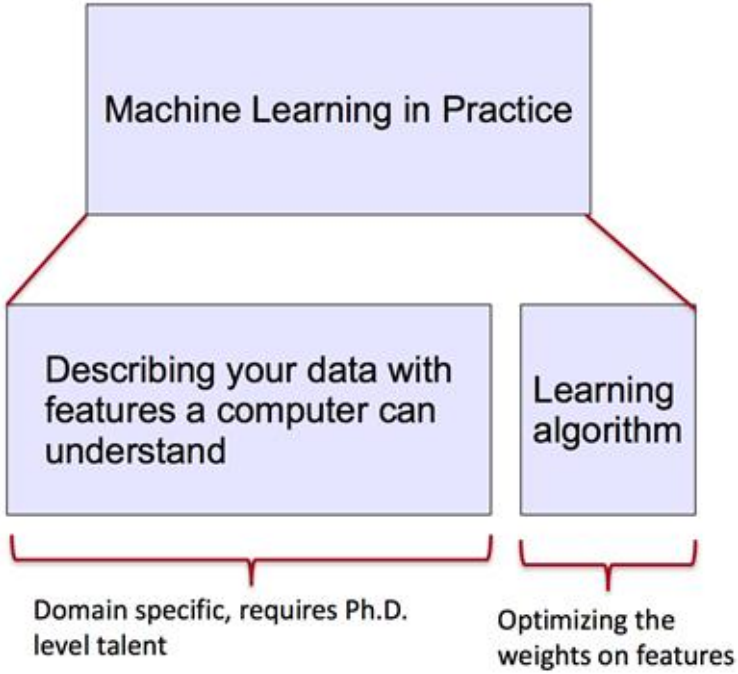
- Idea: use distributed representations
  - Feed-forward NNs for language modeling (predetermined horizon)
  - Recurrent neural networks (infinite horizon) - later

- Learn good representation as a side effect!

  remember self-supervision?

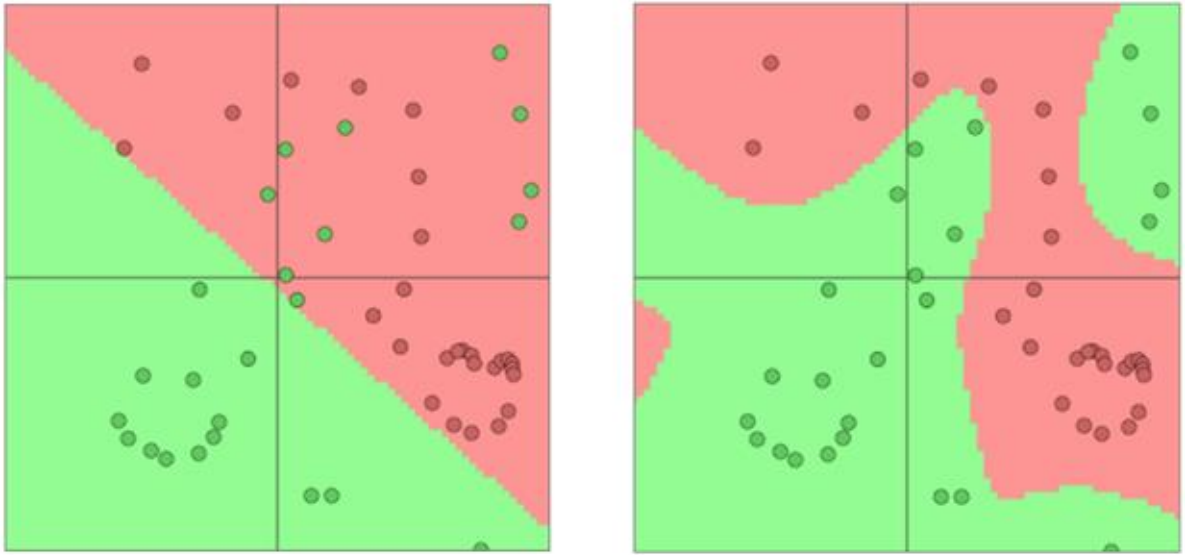
The prominent approach today!

NN motivation 1



Can we learn representations from raw data?

NN motivation 2



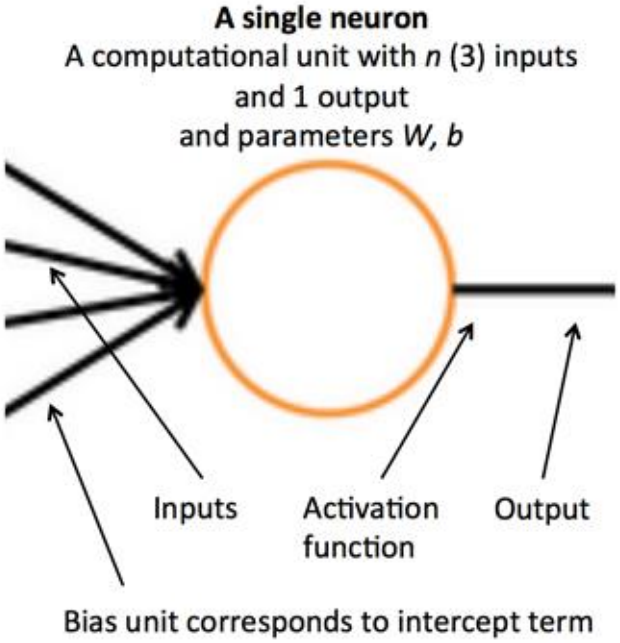
Can we learn non-linear decision boundaries?

NN reminder – single neuron

- A neuron is a computational unit of the form

$$f_{w,b}(x) = f(w^T x + b)$$

- x: input vector
- w: weights vector
- b: bias
- f: activation function (sigmoid, tanh, ReLu, ..)

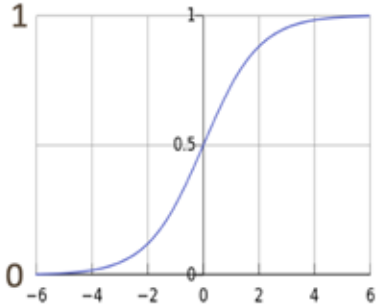


Non-linearities, old and new



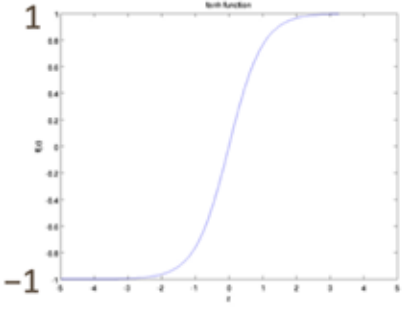
logistic ("sigmoid")

$$f(z) = \frac{1}{1 + \exp(-z)}.$$



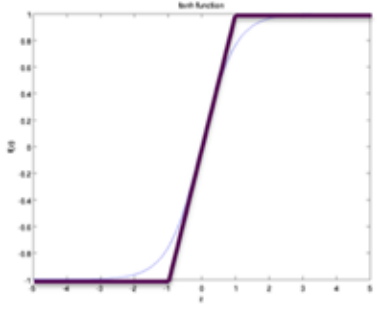
tanh

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}},$$



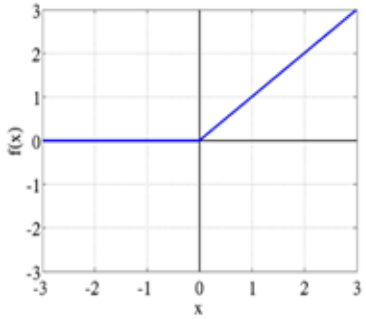
hard tanh

$$\text{HardTanh}(x) = \begin{cases} -1 & \text{if } x < -1 \\ x & \text{if } -1 <= x <= 1 \\ 1 & \text{if } x > 1 \end{cases}$$



ReLU (Rectified Linear Unit)

$$\text{rect}(z) = \max(z, 0)$$



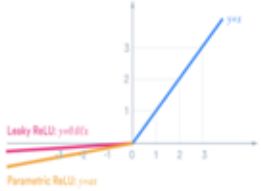
tanh is just a rescaled and shifted sigmoid (2 × as steep, [−1,1]):

$$\tanh(z) = 2\text{logistic}(2z) - 1$$

Both logistic and tanh are still used in various places (e.g., to get a probability), but are no longer the defaults for making deep networks

For building a deep network, the first thing you should try is ReLU — it trains quickly and performs well due to good gradient backflow

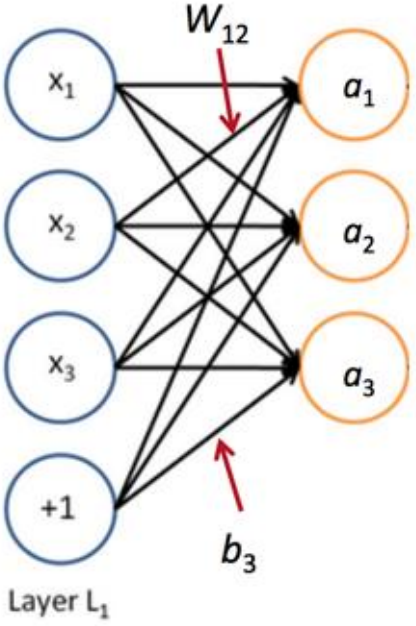
Leaky ReLU / Parametric ReLU



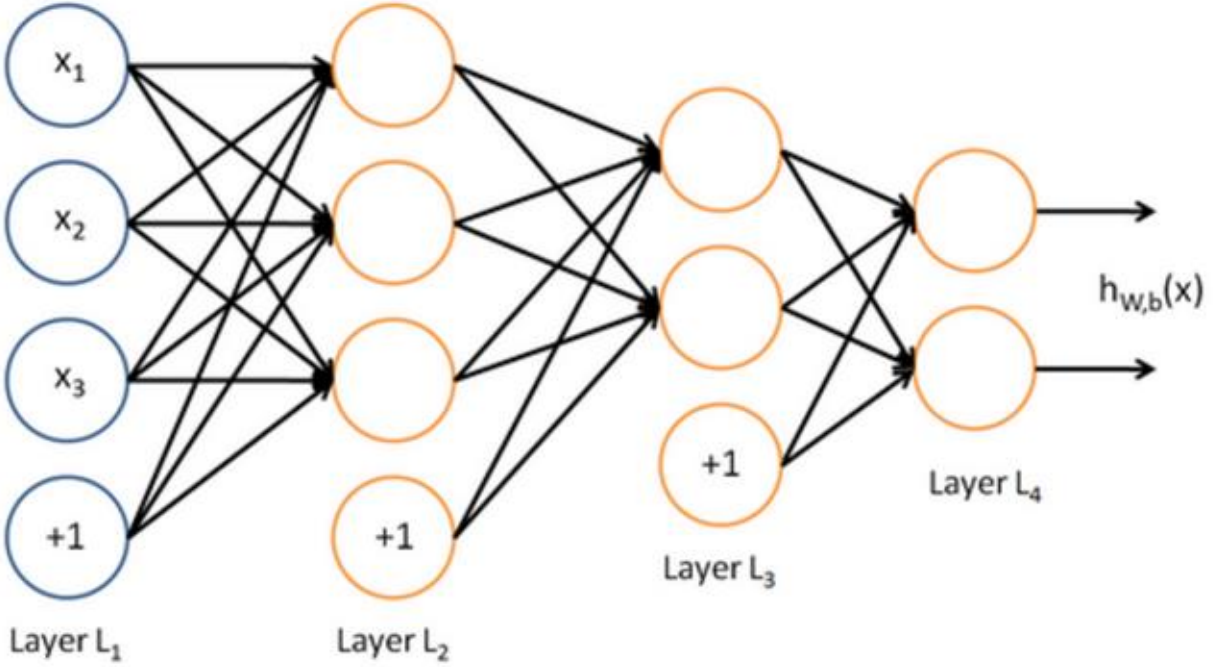
Swish [Ramachandran, Zoph & Le 2017]

NN reminder – matrix notation

$$a_1 = f(W_{11}\,x_1 + W_{12}\,x_2 + W_{13}\,x_3 + b_1)$$
$$a_2 = f(W_{21}\,x_1 + W_{22}\,x_2 + W_{23}\,x_3 + b_2)$$
$$a_3 = f(W_{31}\,x_1 + W_{32}\,x_2 + W_{33}\,x_3 + b_3)$$
$$z = Wx + b, \quad a = f(z)$$
$$x_2 \in \mathbb{R}^4, \quad z \in \mathbb{R}^3, \quad W \in \mathbb{R}^{3\times 4}$$



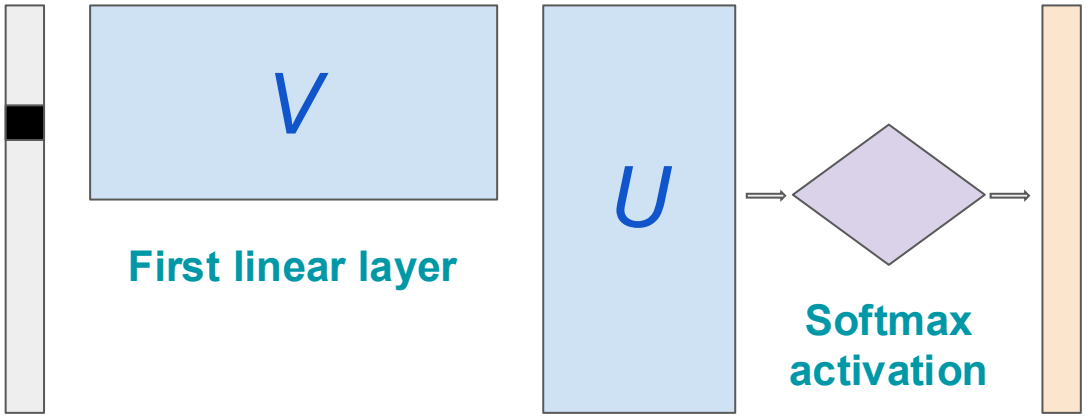
NN reminder – multi-layer network





Reminder: Skipgram: Neural Network

An equivalent way to look at the word2vec model:

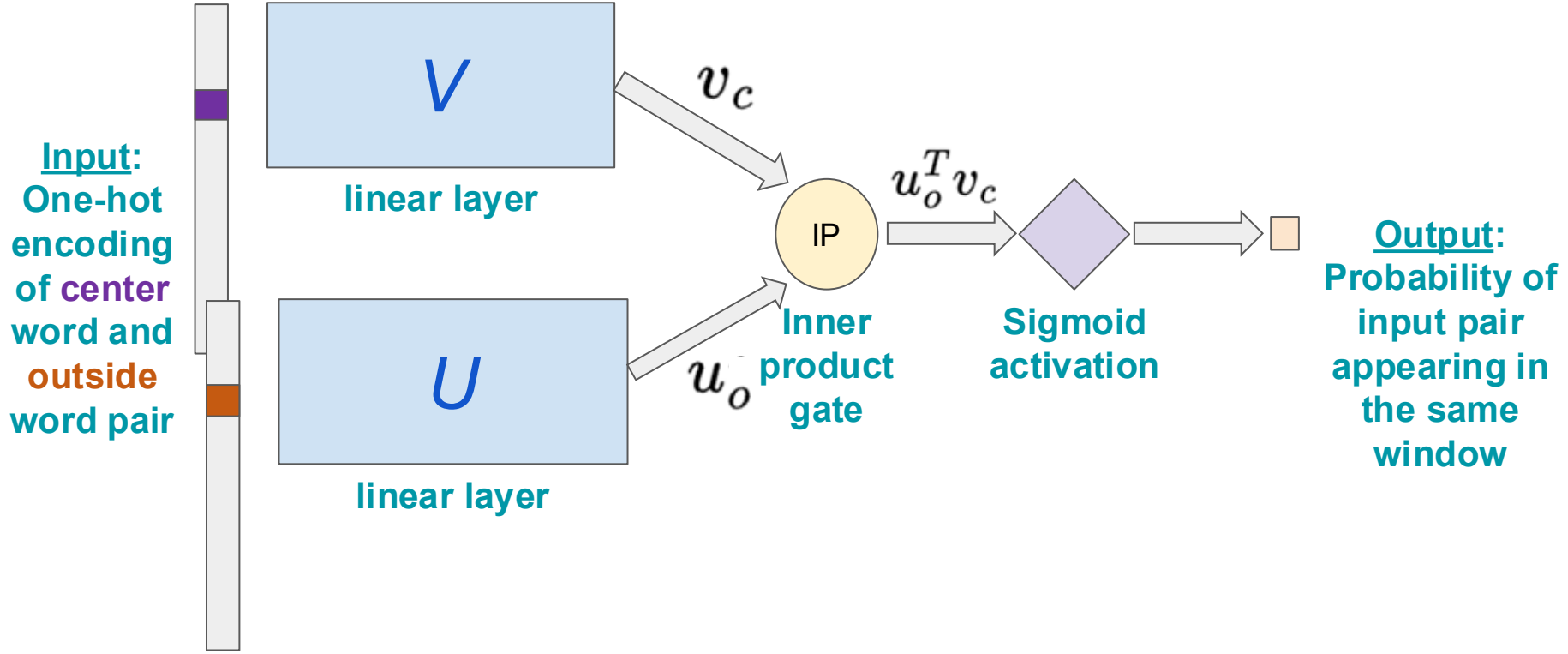


Input:
One-hot encoding of center word

Second linear layer

Output:
Vector of outside word probabilities

Reminder:
Skipgram with Negative Sampling: Neural Network



Bigram LM with feed-forward NNs

- Keep the Markov assumption
- Learn a probability with distributed representations:

$$p(w_i|w_{i-2}, w_{i-1}) =$$

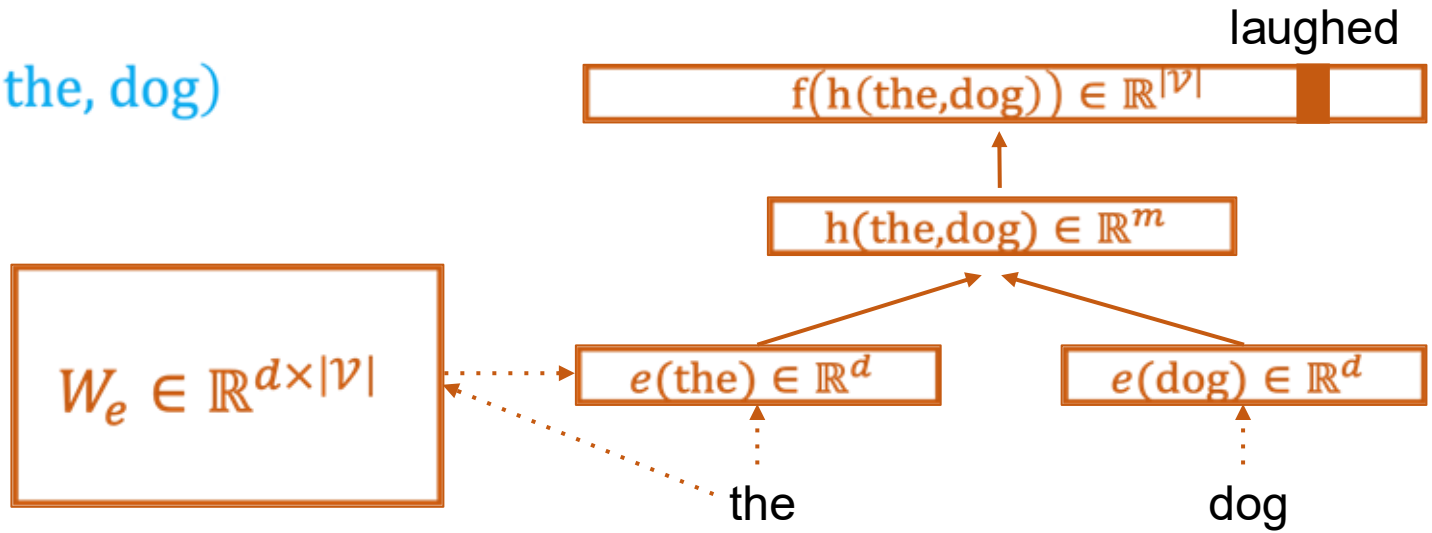


Bigram LM with feed-forward NNs

- Keep the Markov assumption
- Learn a probability with distributed representations:

$$p(w_i|w_{i-2}, w_{i-1}) = f\big(h(w_{i-2}, w_{i-1})\big)$$

$p(\text{laughed} \mid \text{the, dog})$



Bigram LM with feed-forward NNs

- Keep the Markov assumption
- Learn a probability with distributed representations:

$$p(w_i|w_{i-2}, w_{i-1}) = f\big(h(w_{i-2}, w_{i-1})\big)$$

$$e(w) = W_e w, \quad W_e \in \mathbb{R}^{d \times |\mathcal{V}|}, \; w \in \mathbb{R}^{|\mathcal{V}|}$$

$$h(w_{i-2}, w_{i-1}) = \sigma(W_h[e(w_{i-2}); e(w_{i-1})]), \quad W_h \in \mathbb{R}^{m \times 2d}$$

$$f(z) = softmax(W_o z), \; W_o \in \mathbb{R}^{|\mathcal{V}| \times m}, \; z \in \mathbb{R}^m$$

Embedding

Projection

Probability

- Note: concatenate and project == project and sum

Loss function

- Minimize negative log-likelihood

$$\mathcal{L}(\theta) = -\sum_{i=1}^{T} \log p_\theta(w_i | w_{i-2}, w_{i-1})$$

- Train with SGD (remember back-propagation?)
- When we learned about word vectors, we were basically training a language model

Bigram LM with feed-forward NNs

- Keep the Markov assumption
- Learn a probability with distributed representations:

$$p(w_i|w_{i-2}, w_{i-1}) = f\big(h(w_{i-2}, w_{i-1})\big)$$

$$e(w) = W_e w, \quad W_e \in \mathbb{R}^{d \times |\mathcal{V}|}, \; w \in \mathbb{R}^{|\mathcal{V}|}$$

$$h(w_{i-2}, w_{i-1}) = \sigma(W_h[e(w_{i-2}); e(w_{i-1})]), \quad W_h \in \mathbb{R}^{m \times 2d}$$

$$f(z) = softmax(W_o z), \; W_o \in \mathbb{R}^{|\mathcal{V}| \times m}, \; z \in \mathbb{R}^m$$

Embedding

Projection

Probability

Learnable parameters:

Bigram LM with feed-forward NNs

- Keep the Markov assumption
- Learn a probability with distributed representations:

$$p(w_i|w_{i-2}, w_{i-1}) = f\big(h(w_{i-2}, w_{i-1})\big)$$

$$e(w) = W_e w, \quad W_e \in \mathbb{R}^{d\times|\mathcal{V}|}, \; w \in \mathbb{R}^{|\mathcal{V}|}$$

$$h(w_{i-2}, w_{i-1}) = \sigma(W_h[e(w_{i-2}); e(w_{i-1})]), \quad W_h \in \mathbb{R}^{m\times 2d}$$

$$f(z) = softmax(W_o z), \; W_o \in \mathbb{R}^{|\mathcal{V}|\times m}, \; z \in \mathbb{R}^m$$

Embedding

Projection

Probability

Learnable parameters: $d\times|\mathcal{V}| + m\times 2d + |\mathcal{V}|\times m$

We also get word representations, but they may not be of the same quality as those in skip-gram, since some “pressure” is taken of the representations

Bigram LM with feed-forward NNs

- Keep the Markov assumption
- Learn a probability with distributed representations:

$$p(w_i|w_{i-2}, w_{i-1}) = f\big(h(w_{i-2}, w_{i-1})\big)$$

$$e(w) = W_e w, \quad W_e \in \mathbb{R}^{d\times|\mathcal{V}|}, \; w \in \mathbb{R}^{|\mathcal{V}|}$$

$$h(w_{i-2}, w_{i-1}) = \sigma(W_h[e(w_{i-2}); e(w_{i-1})]), \quad W_h \in \mathbb{R}^{m\times 2d}$$

$$f(z) = softmax(W_o z), \; W_o \in \mathbb{R}^{|\mathcal{V}|\times m}, \; z \in \mathbb{R}^m$$

Embedding

Projection

Probability

Learnable parameters: $d\times|\mathcal{V}| + m\times 2d + |\mathcal{V}|\times m$

Linear dependency on the horizon!

Also, each word is processed by different weights

Advantages

- If we see in the training set

  The cat is walking in the bedroom

- We can hope to learn (why?)

  A dog was running through a room

- We don’t care if specific n-grams appeared before
- The number of parameters is O(k) where k is the horizon (k-grams)
  - What did we pay before?
- Complex decision boundaries, scalable, allows over-parameterization

But we still have our Markov assumption

RNN motivation

Apply the same function iteratively, and save the state

